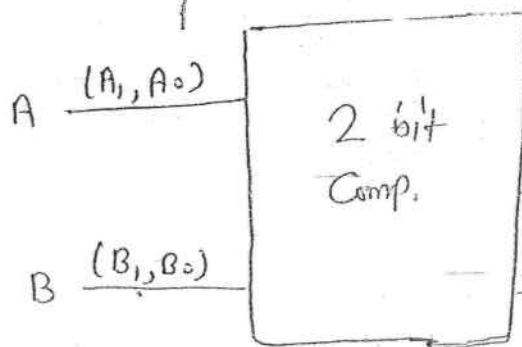


2-bit Comparator

28



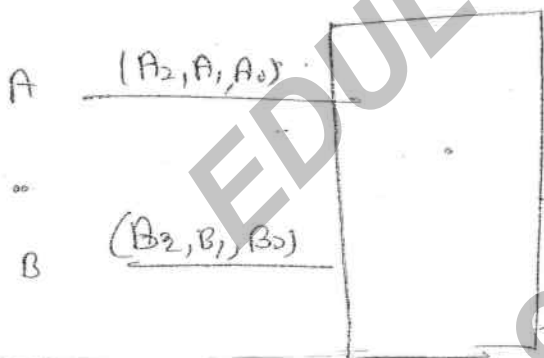
$$A=B \quad x = (A_1 \odot B_1) \cdot (A_0 \odot B_0)$$

$$A>B \quad y = A_1 \bar{B}_1 + (A_1 \odot B_1) (A_0 \bar{B}_0)$$

$$A<B \quad z = \bar{A}_1 B_1 + (A_1 \odot B_1) (\bar{A}_0 B_0)$$

$$y = (A_1 > B_1) + (A_1 = B_1) \cdot (A_0 > B_0)$$

3-bit Comparator



$$A=B \quad x = (A_2 \odot B_2) \cdot (A_1 \odot B_1) \cdot (A_0 \odot B_0)$$

$$A>B \quad y = A_2 \bar{B}_2 + (A_2 \odot B_2) A_1 \bar{B}_1 + (A_2 \odot B_2) (A_1 \odot B_1) A_0 \bar{B}_0$$

$$A<B$$

$$Z =$$

$$Z = (\bar{A}_2 B_2 + (A_2 \odot B_2) \bar{A}_1 B_1 + (A_2 \odot B_2) (A_1 \odot B_1) \bar{A}_0 B_0)$$

Codes

B CD (Binary coded decimal Code):

→ 4 bit code

→ weighted code

→ 8421 code

→ In BCD code each decimal digit is represented with 4 bits.

Decimal

BCD

8421

Excess 3 code

2421

Code

Decimal	BCD	Excess 3 code	2421 Code
0	0000	0011	0000
1	0001	0100	0001
2	0010	0101	0010
3	0011	0110	0011
4	0100	0111	0100
5	0101	1000	0101
6	0110	1001	1011
7	0111	1010	1100
8	1000	1011	1101
9	1001	1100	1110

EDULYF.COM

(Online Store of all study material)

There are 6 invalid BCD codes → 30

→ 1010, 1011, 1100, 1101, 1110, 1111

→ In logic ckt design if any invalid BCD code is present then consider it as don't care.

→ during addition if an invalid BCD code is present 0110 is added to get correct result.

Excess 3 Code

→ Unweighted code

→ To write excess-3 code first write BCD code then add 0011 to each digit.

→ it is self complement code.

→ 2421, 3321, 5211 and 4311 are all called self complement & weighted code.

Gray Code →

→ Unweighted code

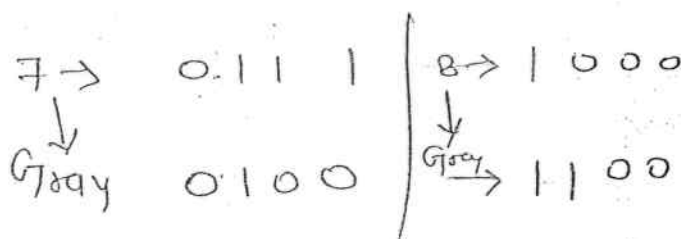
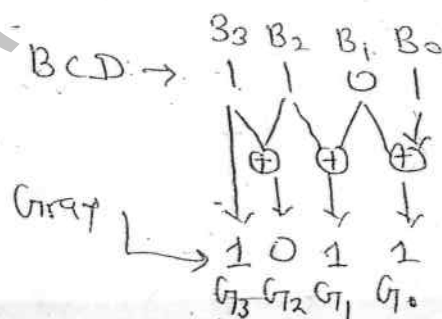
→ In Gray code successive numbers will differ by only one bit.

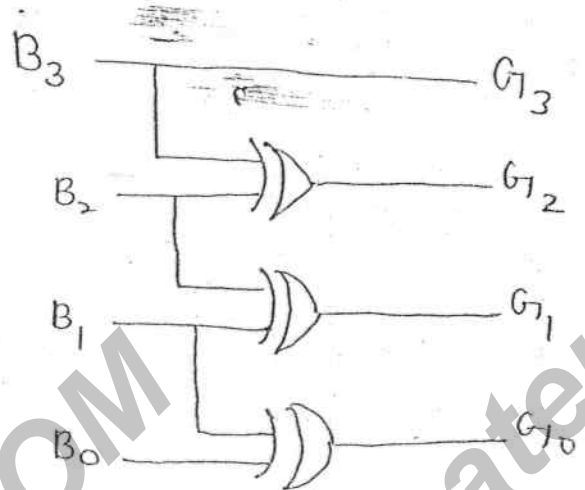
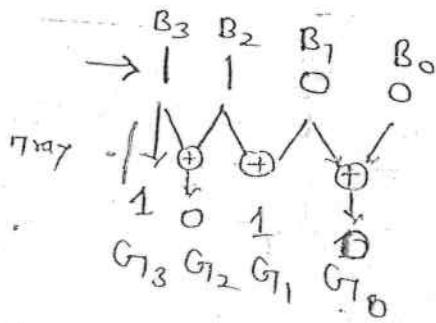
→ Gray code is known as unit distance code (UDC).

→ Cyclic code

→ Reflective code

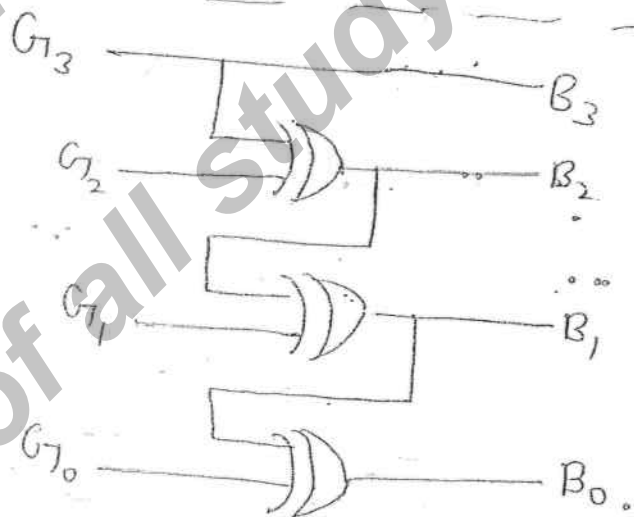
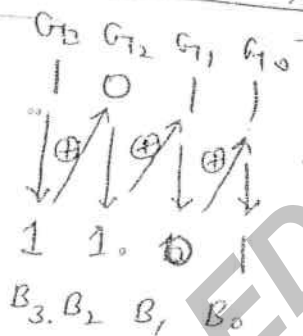
→ Gray code is used in communication to minimize errors.
(minimum error codes).





Binary to Gray Code

Gray to binary



K-map

SOP, POS

→ K-map is a graphical method which can be used to minimize logical expressions with 0, 1, & x.

→ in this, gray code representation is used.

SOP

msb ← A, B ← LSB

	0	1
0	0	1
	00	01
1	2	3
	10	11

msb ← A, B ← LSB

	00	01	11	10
0	0	1	3	2
	000	001	011	010
1	4	5	7	6
	100	101	111	110

31

CD \ AB	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

CD \ AB	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

SOP 2,3,4

Q. minimize exp. for $\bar{A}\bar{B} + AB + \bar{A}B$

B \ A	0	1
0	1	1
1		1

 $\Rightarrow f = \bar{A} + B$
Q. $f(A,B) = \sum m(0,1,2)$

B \ A	0	1
0	1	1
1	1	0

 $\Rightarrow f = \bar{A} + \bar{B}$
Q. $f(A,B) = \sum m(0,1,2,3)$

B \ A	0	1
0	1	1
1	1	1

 $\Rightarrow f = 1$
Q. $f(A,B) = \sum m(1,3) + \sum d(2)$

B \ A	0	1
0	0	1
1	0	1

B \ A	0	1
0		1
1	d	1

Ques $f(A, B) = \sum m(0, 3) + \sum d(2)$

A \ B	0	1
0	1	0
1	X	1

$$f = B \bar{A} + \bar{B}$$

possible value of d is '1'.

Ques

$$f(A, B) = \sum m(0, 2) + \sum d(1, 3)$$

A \ B	0	1
0	1	d
1	1	d

$$f = 1$$

Three Variable $\Rightarrow$

Ques $f(A, B, C) = \sum m(0, 1, 3, 5, 7)$

A \ BC	00	01	11	10
0	1	1	1	
1		1	1	

$$\Rightarrow f = C + \bar{A}\bar{B}$$

Ques

$$f(A, B, C) = \sum m(0, 1, 2, 4, 7)$$

A \ BC	00	01	11	10
0	1	1		1
1	1		1	

$$f = \bar{A}\bar{B} + \bar{A}\bar{C} + \bar{B}\bar{C} + ABC$$

Ques

$$f(A, B, C) = \sum m(1, 3, 5, 7)$$

A \ BC	00	01	11	10
0		1	1	
1		1	1	

$$f = C$$

Ques

$$f(A, B, C) = \sum m(1, 3, 6, 7)$$

A \ BC	00	01	11	10
0		1	1	
1			1	1

$$\Rightarrow AB + \bar{A}C$$

Procedure

32

- (i) octet (group of 8 cells)
- (ii) quad
- (iii) pair
- (iv) single term
- (v) Remove Redundent term.

Implicants $\rightarrow$ No. of minterms

Prime implicants = 3 (ex. $\leftarrow$ last page) $AB, \bar{A}C, BC$

Essential prime implicants = 2, $AB, \bar{A}C$ (EPI)

- $\rightarrow$ Each min term is known as implicants.
- $\rightarrow$ Prime implicant is a product term which is obtained by combining maximum possible adjacent cells.
- $\rightarrow$ EPI is a prime implicant which is obtain by combining in one way and no alternative way.

Ques $f(A, B, C) = \sum m(0, 1, 5, 6, 7)$

BC \ A	00	01	11	10
0	1	1		
1		1	1	1

$$f_1 = \bar{A}\bar{B} + \bar{B}C + AB$$

$$\text{OR } f_2 = \bar{A}\bar{B} + \bar{A}C + AB$$

$\Rightarrow$ K-map will provide always minimized expression but not necessarily unique solution.

$$I_{\text{implicant}} = 5$$

$$PI = 4$$

$$EPI = 2$$

Q.2

$$f(A, B, C) = \sum m(1, 3, 6, 7)$$

$$\sum m(0, 1, 2, 5, 6, 7)$$

A \ BC				
	00	01	11	10
0	1	1		1
1		1	1	1

$$f = \bar{A}\bar{B} + AC + A\bar{C}$$

OR

1	1		
	1	1	1

A \ BC				
	00	01	11	10
0	1	1		1
1		1	1	1

$$f = \bar{A}\bar{C} + \bar{B}C + AB$$

$$I = 6$$

$$PI = 6$$

$$EPI = 0$$

Q.3

$$f(A, B, C) = \sum m(0, 1, 3, 4, 5)$$

A \ BC				
	00	01	11	10
0	1	1	1	
1	1	1		

$$f = \bar{B} + \bar{A}C$$

both terms covering

$$EPI = 2$$

$$I = 5$$

$$PI = 2$$

$$EPI = 2$$

Q. $f(A, B, C) = \sum m(1, 3, 5, 7) + \sum d(6)$ 33

	1	1	1	
	1	1	1	

$f = C$

Q. $f(A, B, C) = \sum m(0, 1, 5, 7) + \sum d(2, 4, 6)$

	BC	00	01	11	10
A	0	1	1		X
	1	X	1	1	X

$f = A + \bar{B}$

Q. $f(A, B, C) = \sum m(0, 1, 6, 7) + \sum d(3, 5)$

	BC	00	01	11	10
A	0	1	1	1	1
	1			1	1

$f = \bar{A}\bar{B} + AB$

redundant

	BC	00	01	11	10
A	0	1	1		
	1	X	X	1	1

$f = \bar{B} + A$

⇒ 4 Variable

Q. $f(A, B, C, D) = \sum m(0, 1, 3, 5, 7, 8, 9, 11, 13, 15)$

	CD	00	01	11	10
AB	00	1	1	1	
	01		1	1	
	11	1	1	1	
	10	1	1	1	

$f = \bar{A}\bar{B}\bar{C} + AB\bar{C} + D$
 $= D + \bar{B}\bar{C}$

	CD	00	01	11	10
AB	00	1	1	1	
	01		1	1	
	11		1	1	
	10	1	1	1	

Ques $f(A, B, C, D) = \sum m(0, 1, 4, 5, 8, 9, 13, 15)$

AB \ CD	00		01		11		10	
	00	01	11	10	00	01	11	10
00	1	1						
01	1	1						
11			1	1				
10	1	1			1			

AB \ CD	00		01		11		10	
	00	01	11	10	00	01	11	10
00	1	1						
01	1	1						
11			1	1				
10	1	1			1			

Ques 22 page No. 14

$f = \overline{A}\overline{C} + \overline{B}\overline{C} + ABD$

X	Y	Z	f
0	0	0	X
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	X

X \ YZ	00		01		11		10	
	00	01	11	10	00	01	11	10
0					1	1		
1	1	1			X			

$f = \overline{X}Y + X\overline{Y}$

$\Rightarrow$ POS

$f(A, B) = \prod m(0, 1, 3)$

A \ B	0		1	
	0	1	0	1
0	1	1	1	1
1	1	1	1	1

$f = A \cdot \overline{B}$

Ques $f(A, B, C) = \prod m(0, 1, 3, 5, 7)$

A \ BC	00		01		11		10	
	00	01	11	10	00	01	11	10
0	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1

$f = (A+B) \cdot \overline{C}$

Ques for the given K-map minimize POS expression is

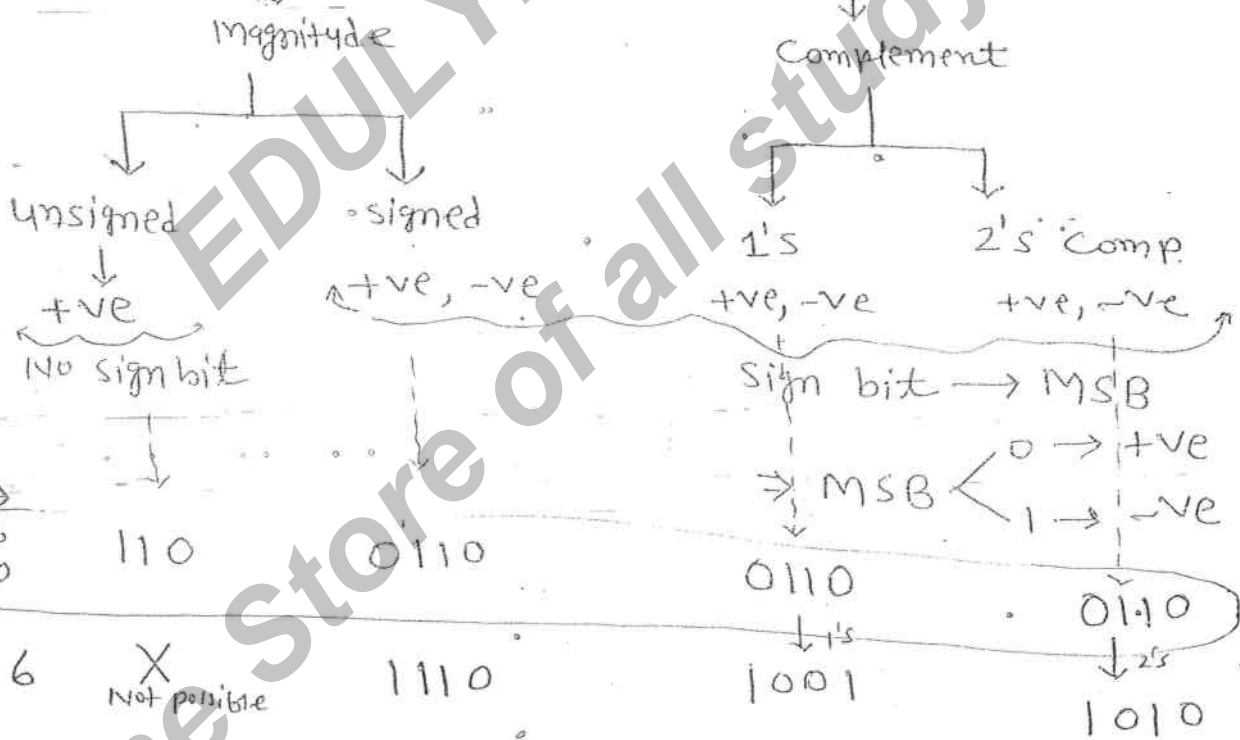
	BC			
	00	01	11	10
A				
0	0	X	1	X
1	X	0	1	0

34

Soln for pos $\rightarrow$ Remove 1's } for sop $\rightarrow$ Remove 0's

$$f = B \cdot C$$

$\Rightarrow$ Data Representation $\Rightarrow$



Unsigned
+ve, 4 bit

0000 min 1111 max

$\rightarrow$ in unsigned magnitude the Range of Inters with n bits is 0 to $(2^n - 1)$.

Signed magnitude Representation

- it is for both +ve & -ve Numbers.
- In this +ve Numbers are represented as Normal binary with MSB is zero '0'
- In -ve NO.s are represented as normal binary with MSB '1'.

ex → +13 01101
 -13 11101

ex → given sign magnitude

Sign magnitude	Decimal
0 0 0 0	+0
0 0 0 1	+1
1 1 1 1	...
0 1 1 0	+6
0 1 1 1	+7
1 0 0 0	...
1 0 0 1	-0
1 0 0 0	-1
1 1 1 0	-6
1 1 1 1	-7

4 bit Range -7 to +7

Range of n bit Np. = $-(2^{n-1}-1)$ to $+(2^{n-1}-1)$
(signed no.)

ex → 8 bit → Range -127 to +127

1's Complement Representation :-

35

- it can be used for both +ve & -ve numbers.
- In this +ve No.'s are represented as Normal Binary with MSB is '0'.
- To represent -ve No. there are two steps

Step 1 → write +ve No.

Step 2 → 1's Complement of it.

ex +13 $\Rightarrow$ 01101

-13 $\Rightarrow$ 10010

0000 $\rightarrow$ +0

0001 $\rightarrow$ +1

0010 $\rightarrow$ +2

0011 $\rightarrow$ +3

0100 $\rightarrow$ +4

0101 $\rightarrow$ +5

0110 $\rightarrow$ +6

0111 $\rightarrow$ +7

1000 $\rightarrow$ -7

1001 $\rightarrow$ -6

1010

1011 $\rightarrow$ -4

1100 $\rightarrow$ -3

1101 $\rightarrow$ -2

1110 $\rightarrow$ -1

1111 $\rightarrow$ -0

Range

$$\Rightarrow -\left(2^{n-1}-1\right) \text{ to } \left(2^{n-1}-1\right)$$

* Ques → perform 5-4 operation in 1's complement operation.

Sol $\Rightarrow +5 + (-4) \Rightarrow$

$$\begin{array}{r} 0101 \\ + 1011 \\ \hline \end{array}$$

+4 = 0100

$\Rightarrow -4 = 1011$

10000

← EAC

Final answer

Disadvantages of sign magnitude & 1's complement Representation

- (i) There are two representations with present for zero.
- (ii) During addition: if any carry (EAC) is present then it is added to LSB to get correct result.
- To overcome these disadvantages 2's Complement Representation is used.

2's Complement Representation

- This is used for both +ve & -ve Numbers.
- +ve No. is represented with as Normal binary with '0'.
- -ve No. Representation

Step 1 → write +ve No.

Step 2 → take 2's complement.

ex → +7 → 0111
-7 → ~~1111~~ 1001

Ques → a No. 0 is Represented in 2's comp. form as 1001
eqv. decimal = -7

Ans → In 2's comp. form → ~~1111~~ 11001
eqv. decimal = -7

→ in 2's complement to extend No. of bits
Copy MSB bit.

2's Complement

decimal

36

0 0 0 0
0 0 0 1

+0
+1

0 1 1 0

+6

0 1 1 1

+7

gmp

* → 1 0 0 0 →
1 0 0 1 →

-8
-7

1 1 1 0 →

-2

1 1 1 1 →

-1

Range of 2's Complement = $-(2^{n-1})$ to $(2^{n-1} - 1)$

ex → 4 bit → -8 to 7

Ques perform 5-4 using 2's complement

+5 → 0 1 0 1

+4 → 0 1 0 0

-4 → 1 1 0 0

0 1 0 1

1 1 0 0

① 0 0 0 1 ⇒ +1

Discard

Ques 5+4 using 2's

0 0 1 0 1

0 0 1 0 0

0 1 0 0 1

(over flow)

→ ① 0 1 0 0 1

1 0 0 1 → -7

→ (+9)

→ overflow may occur when two same sign no. are added to detect overflow there are two methods

(i) if x, y are is sign bit of ~~both~~ numbers ~~not~~ same then

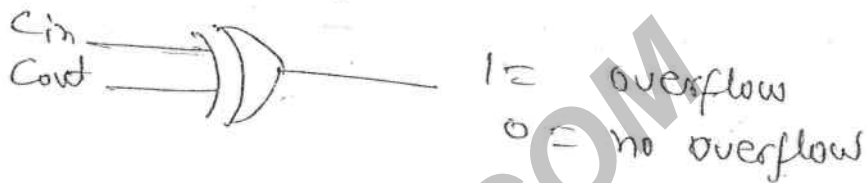
Condition for overflow

$$\bar{X}\bar{Y}Z + XY\bar{Z}$$

31/08/2011

if

Carry into the MSB is C_{in} and carry in out from it is C_{out} and if the two are different overflow occurs.



31/08/2011

Ques

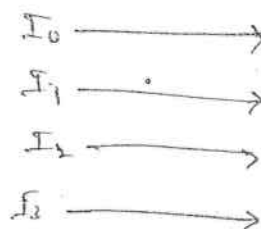
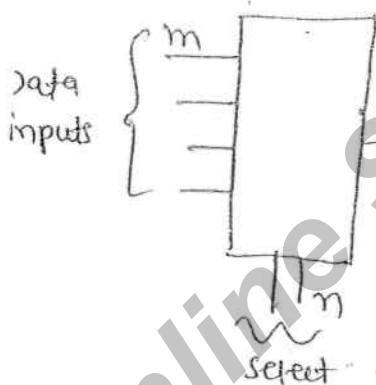
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

SOP

$$Y = \bar{A}\bar{B}.1 + \bar{A}B.0 + A\bar{B}.1 + ABC$$

$$Y_{POS} = (A+B+1) \cdot (A+B+0) \cdot (A+B+1) \cdot (\bar{A}+\bar{B}+C)$$

Multiplexer : $\Rightarrow$

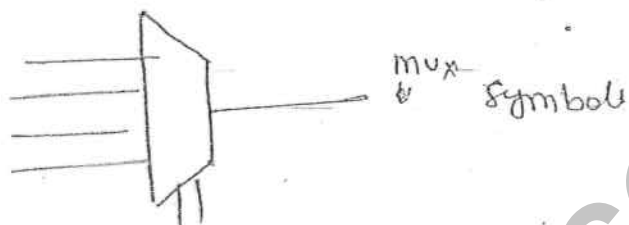


$$m = 2^n$$

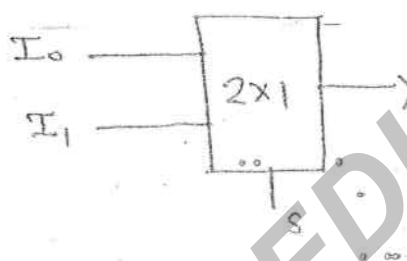


- $\Rightarrow$ Multiplexer is a Combinational ckt which have many data inputs & single output depending on select (or) control one of the data input is transferred to the output, Hence it is known as many to one ckt.
- Other names of mux $\rightarrow$
- $\rightarrow$ many to one input $\rightarrow$ parallel to serial data converter
 - $\rightarrow$ Data selector
 - $\rightarrow$ Universal logic ckt

if m is data input & n is select inputs then
 the relation b/w them is $m = 2^n \Rightarrow n = \log_2 m$

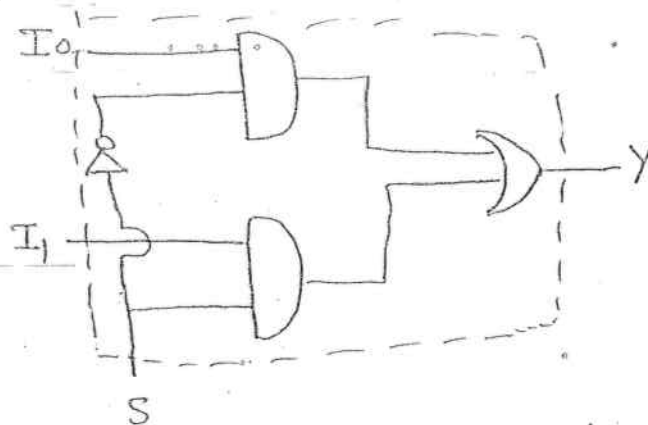


2 to 1 mux (2:1 mux)



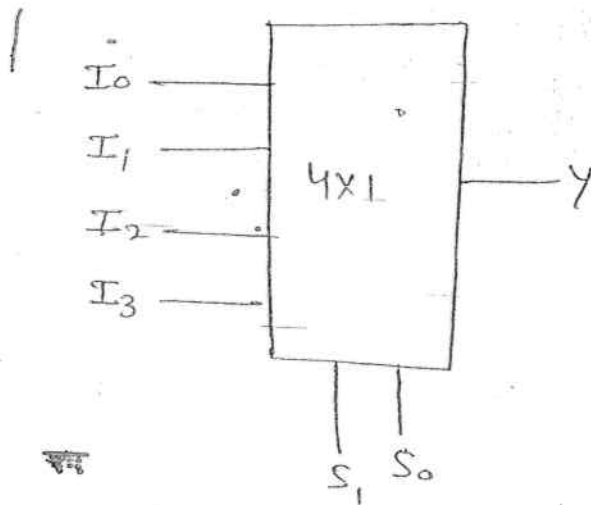
S select line	Y output
0	I_0
1	I_1

$$Y = \bar{S}I_0 + SI_1$$



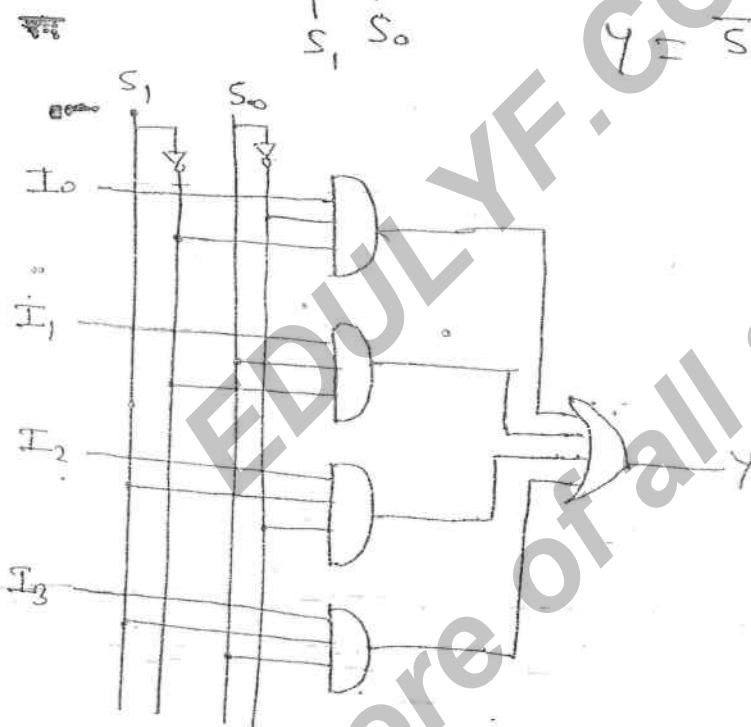
$\Rightarrow$ mux basically contains two level
AND-OR network inside.

4x1 mux $\Rightarrow$



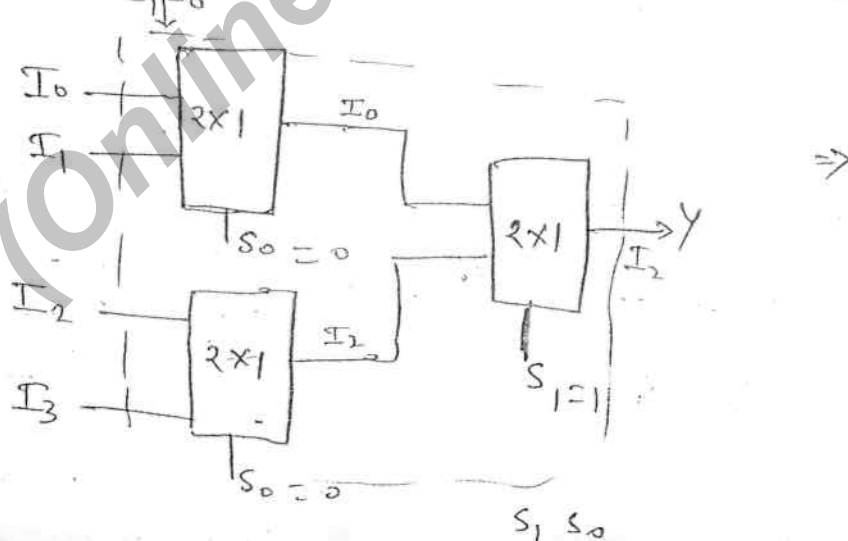
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

$$Y = \bar{S}_1 \bar{S}_0 I_0 + \bar{S}_1 S_0 I_1 + S_1 \bar{S}_0 I_2 + S_1 S_0 I_3$$



(i) Implementation of Higher order multiplexers using lower order

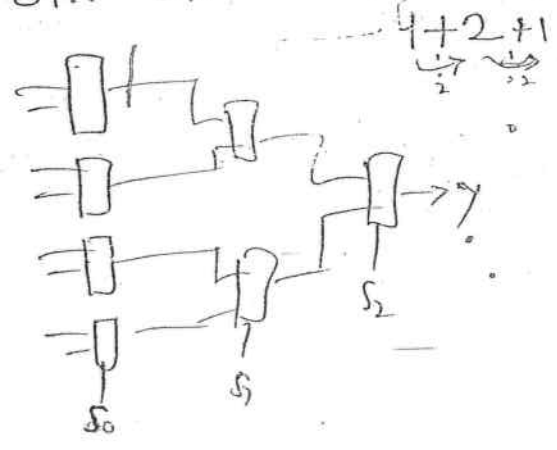
(1) $4 \times 1 \text{ mux} \rightarrow 2 \times 1 \text{ mux} \Rightarrow$



ex $\rightarrow 10 \Rightarrow I_2$

8.7. 8x $\xrightarrow{\quad} 2 \times 1 \text{ mux (7)}$

38



$$16 \times 1 \xrightarrow[8+4+2+1]{18} 2 \times 1$$

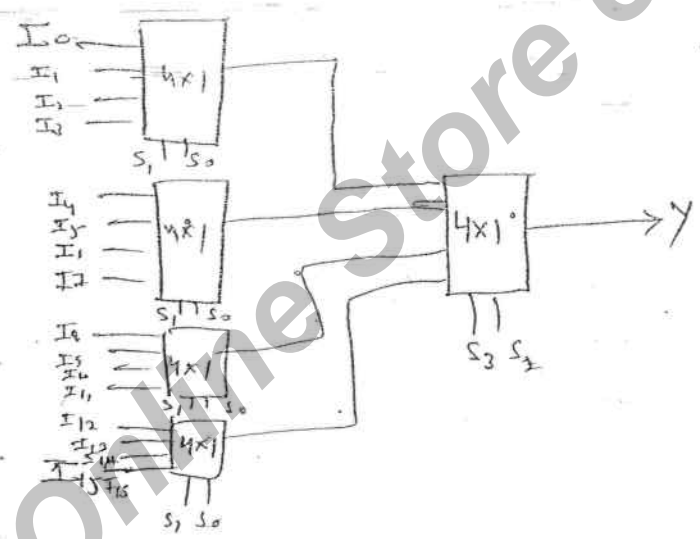
$$64 \times 1 \xrightarrow{63} 2 \times 1$$

$$256 \times 1 \xrightarrow{255} 2 \times 1$$

$$2^n \times 1 \xrightarrow{2^n - 1} 2 \times 1$$

8.8

$$16 \times 1 \text{ mux } \xrightarrow[4+1=5]{s_3 s_2 s_1 s_0} 4 \times 1$$



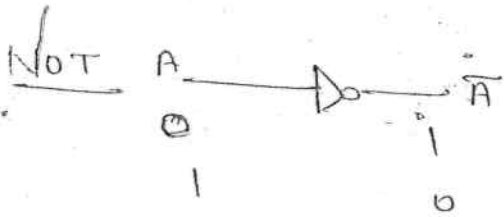
2x $\rightarrow$ $s_3 s_2 s_1 s_0$ $s_3 s_2$ I_{11}

$$64 \times 1 \text{ mux } \xrightarrow[11+4+1]{21} 4 \times 1$$

$$64 \times 1 \xrightarrow[8+1]{9} 8 \times 1$$

$$256 \times 1 \xrightarrow[16+1]{17} 16 \times 1$$

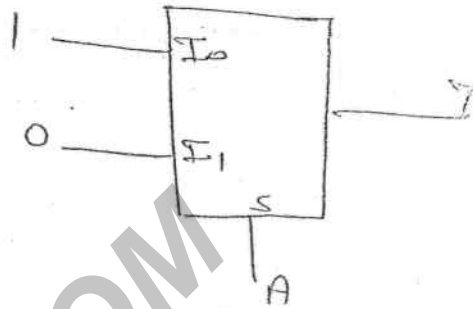
1) MUX AS Universal logic circuit :



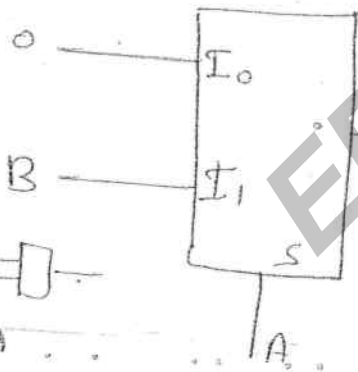
$$Y = \bar{S}I_0 + SI_1$$

$$Y = \bar{A} \cdot 1 + A \cdot 0$$

$$Y = \bar{A}$$



Implement 2 input AND gate using 2x1 mux $\Rightarrow$

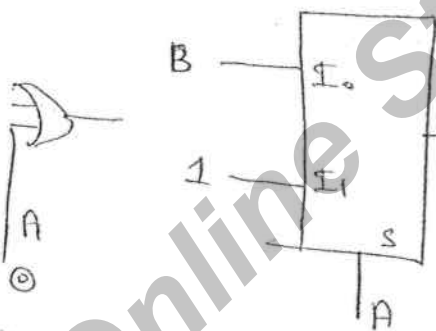


$$Y = \bar{S}I_0 + SI_1$$

$$= \bar{A} \cdot 0 + A \cdot B$$

AND $\rightarrow$ NAND

Implement 2 input OR gate using 2x1 mux



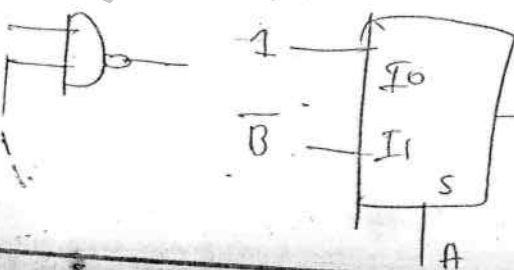
$$Y = \bar{S}I_0 + SI_1$$

$$= \bar{A}B + A \cdot 1$$

$$= A + B$$

NAND

$$\text{NAND} = \text{N_AND}$$



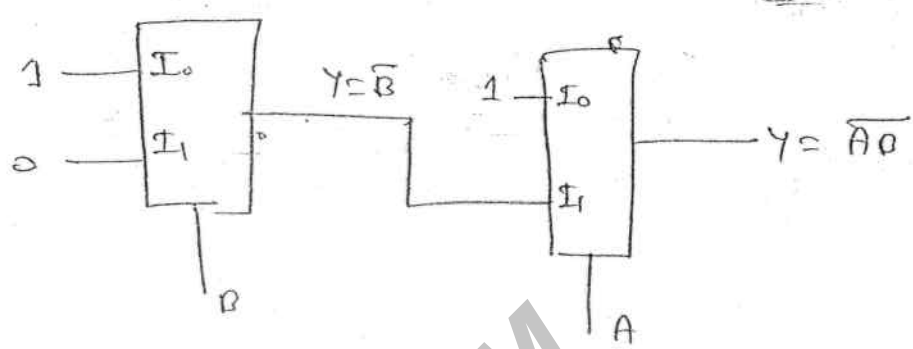
$$Y = \bar{S}I_0 + SI_1$$

$$= \bar{A} \cdot 1 + A\bar{B}$$

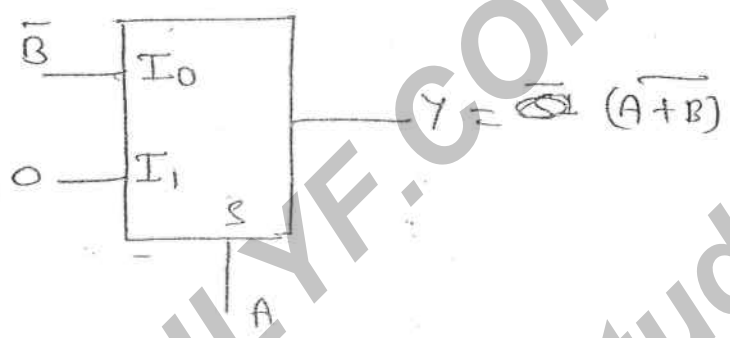
$$= \bar{A} + \bar{B}$$

$$= \overline{AB}$$

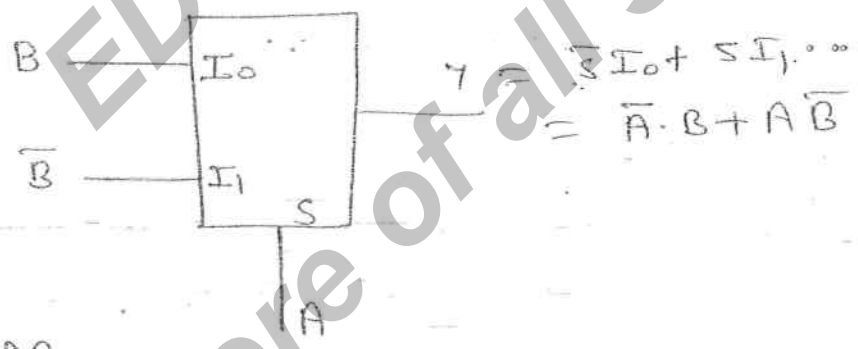
pr) NAND
 $\downarrow$
 2 $\rightarrow$ 2x1 mux



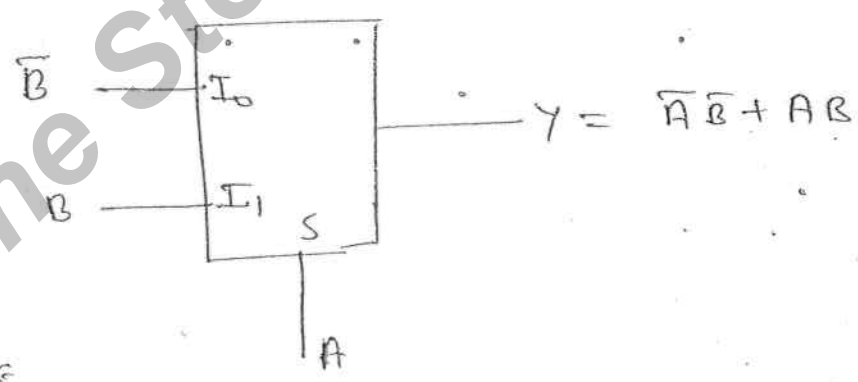
NOR
 $\downarrow$
 2 TWO
 2x1 mux



EX-OR

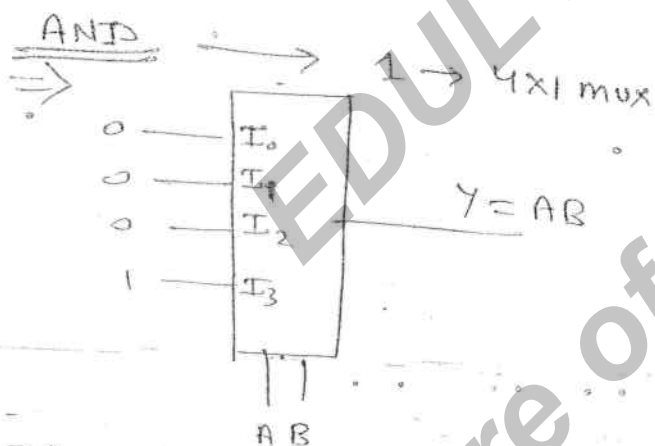
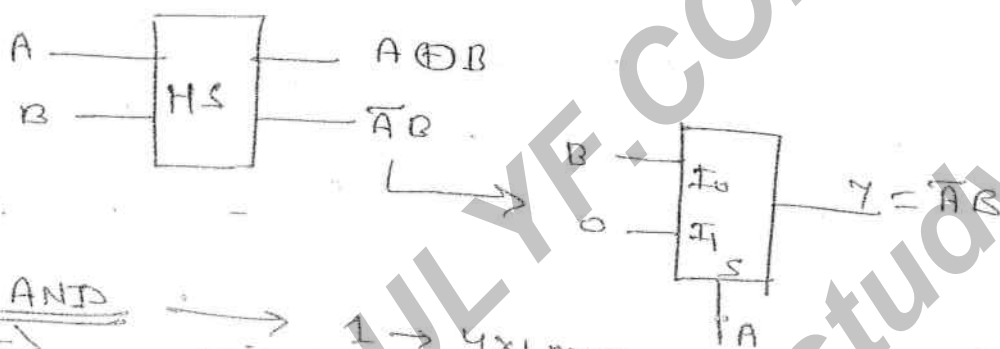
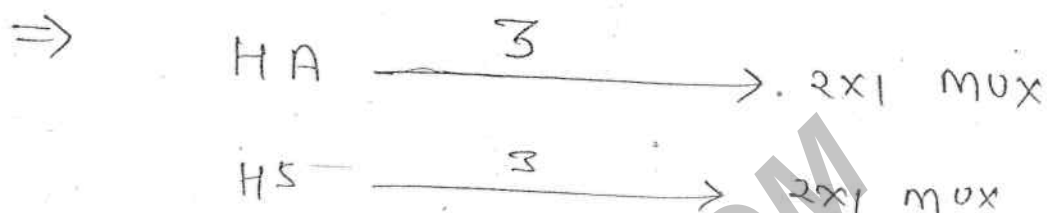


EX-NOR



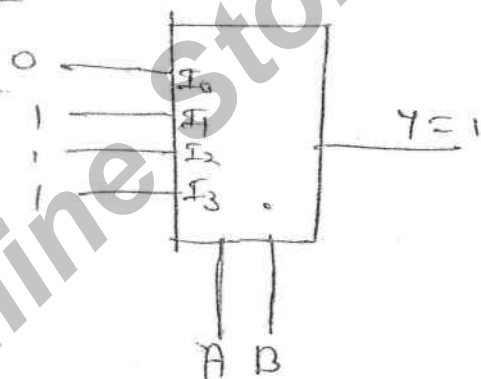
LOW GATE

Ques. to implement two input Ex-OR gate and AND gate minimum No. of 2:1 mux required are respectively Ans $\rightarrow 2, 1$



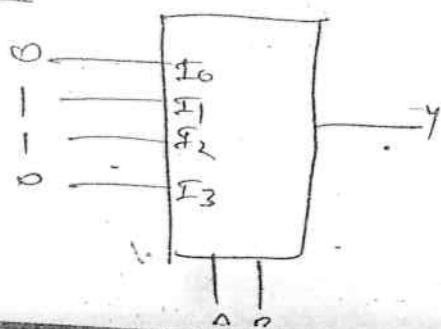
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR gate

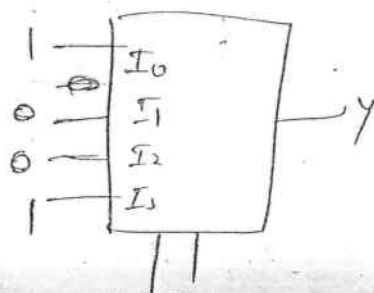


A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

EX-OR



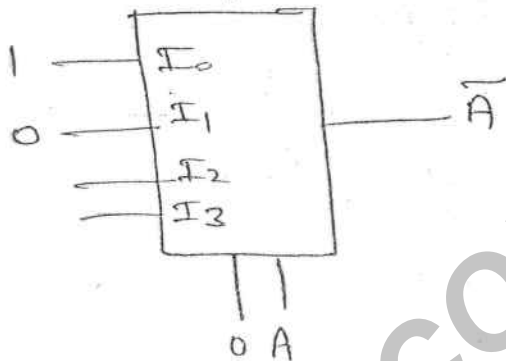
EX-NOR



HA $\xrightarrow{2}$ 4x1

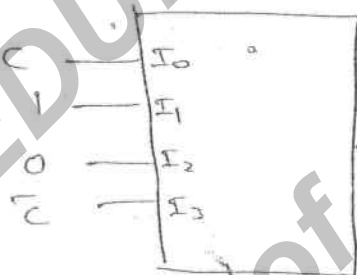
40

NOT gate



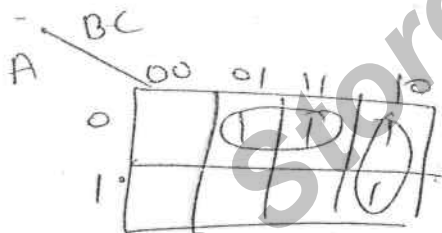
(III) minimization of given logic circuit $\Rightarrow$

Ques for the given mux ckt minimized logical expression is,



for conventional

$$f = \bar{S}_1 \bar{S}_0 I_0 + \bar{S}_1 S_0 I_1 + S_1 \bar{S}_0 I_2 + S_1 S_0 I_3$$



$$f = \bar{A}C + B\bar{C}$$

$$f = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$$

$$= \bar{A}\bar{B}C + \bar{A}B + A\bar{B}\bar{C}$$

$$= \bar{A}\bar{B}C + B(\bar{A} + \bar{C})$$

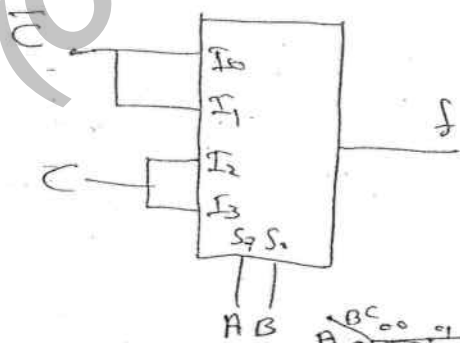
$$= \bar{A}B + B\bar{C} + \bar{A}\bar{B}\bar{C}$$

$$= \bar{A}(B + \bar{C}) + B\bar{C}$$

$$= \bar{A}B + \bar{A}\bar{C} + B\bar{C}$$

$$= \bar{A}(B + \bar{C}) + B\bar{C}$$

Ques minimize



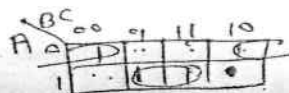
$$f = \bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + A\bar{B}C + ABC$$

$$f = \sum m(0, 2, 5, 7)$$

$$f = \bar{A}\bar{C}(B + \bar{B}) + AC(B + \bar{B})$$

$$= \bar{A}\bar{C} + AC$$

$\Rightarrow$ Ex-NOR

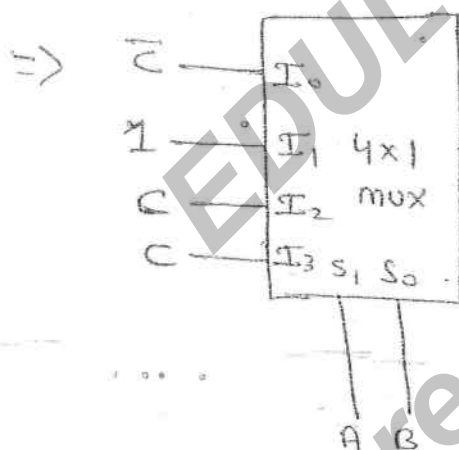


(iv) Ques implement $f(A, B, C) = \sum m(0, 2, 3, 5, 7)$ using 4×1 mux with AB as select. line.

Solⁿ

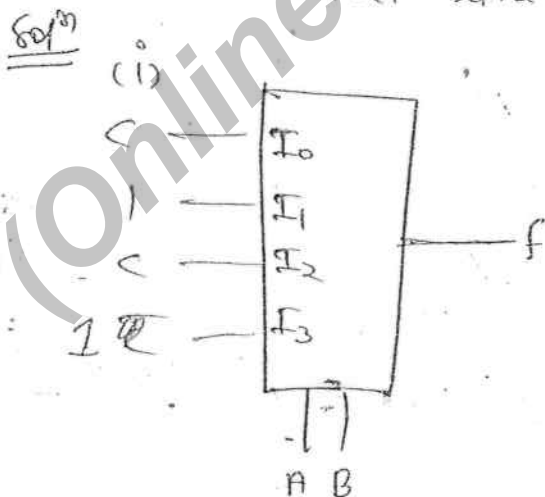
Dec.	A	B	C	$\bar{C}$	C
0	0	0	0	$\bar{C}$	C
1	0	0	1	$\bar{C}$	C
2	0	1	0	$\bar{C}$	C
3	0	1	1	$\bar{C}$	C
4	1	0	0	$\bar{C}$	C
5	1	0	1	$\bar{C}$	C
6	1	1	0	$\bar{C}$	C
7	1	1	1	$\bar{C}$	C

	I_0	I_1	I_2	I_3
$\bar{C}$	0	2	4	6
C	1	3	5	7
$\bar{C}$	1	1	C	C



$\Rightarrow$ One 4×1 mux and one NOT gate Required to implement f .

Ques implement $f(A, B, C) = \sum m(1, 2, 3, 5, 6, 7)$, using 4×1 mux with (i) AB as select line (ii) AC as select line (iii) BC as select line.



	I_0	I_1	I_2	I_3
$\bar{C}$	0	2	4	6
C	1	3	5	7
C	1	1	C	C

$\Rightarrow$ one AND gate



ing me 4

any two variable functions
 can implement some of 3 variable functions.

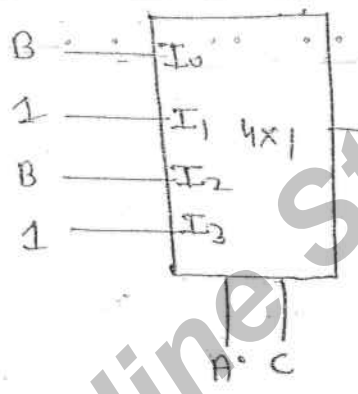
41

one 4x1 mux & one NOT gate } any two variable function
 any three variable function.

one 8x1 mux } any three variable
 some 4 variable

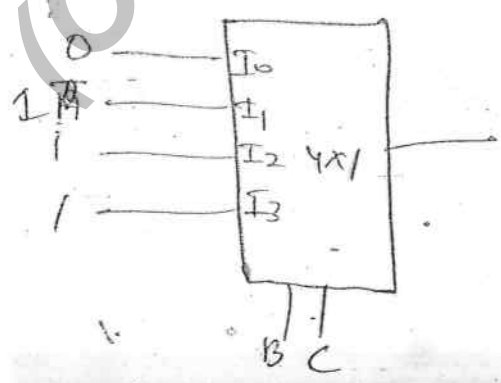
one 8x1 mux & NOT gate } any three var
 any 4 var

(ii) AC AS select line



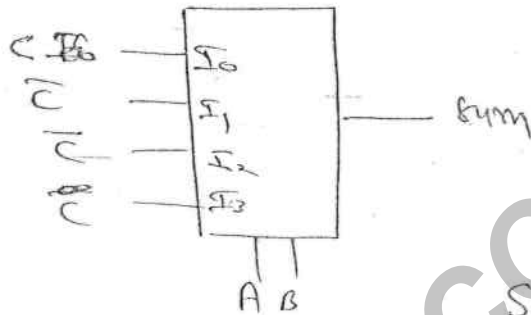
	I ₀	I ₁	I ₂	I ₃
$\bar{B}$	0	①	4	⑤
B	②	③	⑥	⑦
	B	1	B	1

(iii) BC AS select line



	I ₀	I ₁	I ₂	I ₃
$\bar{A}$	0	①	②	③
A	4	⑤	⑥	⑦
	0	1	1	1

One $\Rightarrow$ To implement sum output of full adder using 4×1 mux shown in figure. I_0, I_1, I_2, I_3 respectively are $\rightarrow$



$$\text{sum} = \sum m(1, 2, 4, 7)$$

Sum $\Rightarrow$ one 4×1 mux & NOT gate

$\Rightarrow$

	I_0	I_1	I_2	I_3
$\bar{A}$	0	2	4	6
A	1	3	5	7

for carry $\Rightarrow$ one 4×1 mux $\sum m(3, 5, 6, 7)$

$\Rightarrow$ FA $\Rightarrow$ Two 4×1 mux & NOT gate
(1 for carry, 1 for sum)

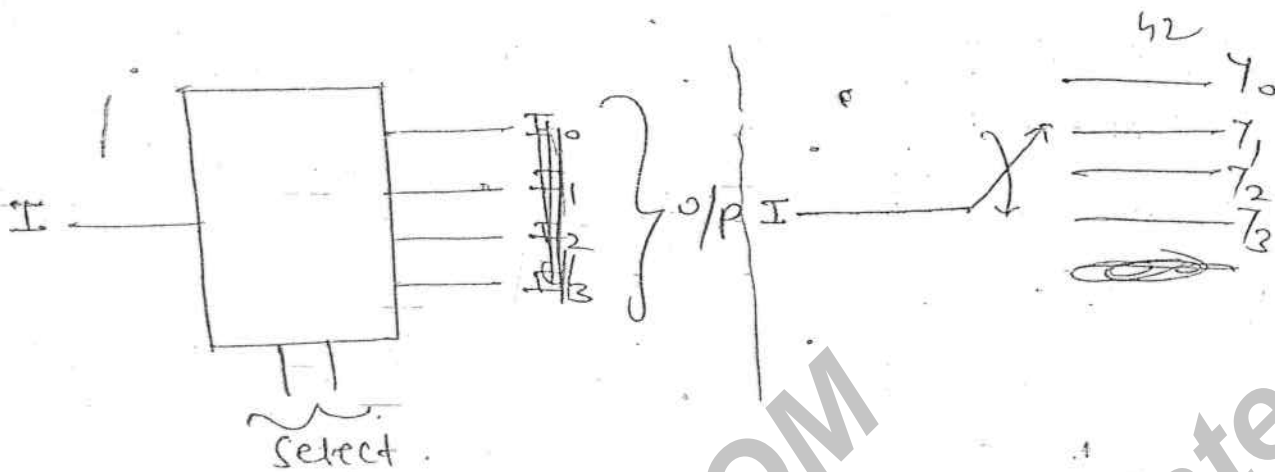
$\Rightarrow$ FA $\xrightarrow{2}$ 8×1 mux

$\Rightarrow$ DE MULTIPLEXER $\Rightarrow$

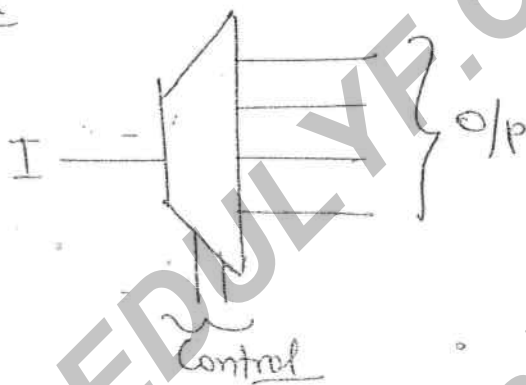
$\rightarrow$ De mux is a combinational ckt which have one input and many outputs.

$\rightarrow$ Depending on select (OR) control input is transferred to one of the output line hence it is known as $\rightarrow$ one to many ckt.

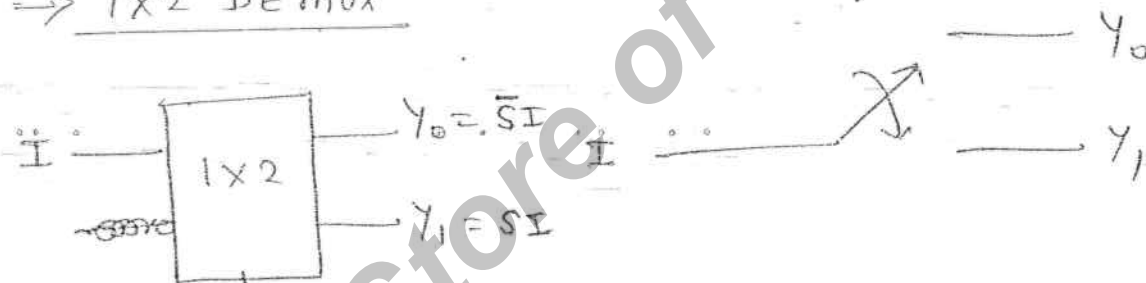
$\rightarrow$ Data Distributor



Symbol



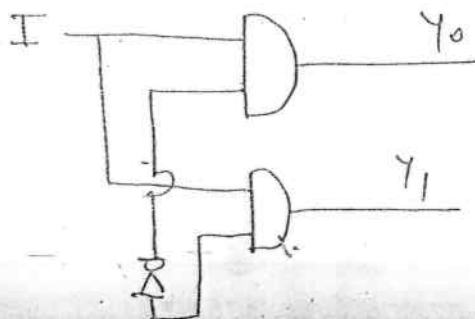
⇒ 1X2 DE mux



S	Y ₁	Y ₀
0	0	I
1	I	0

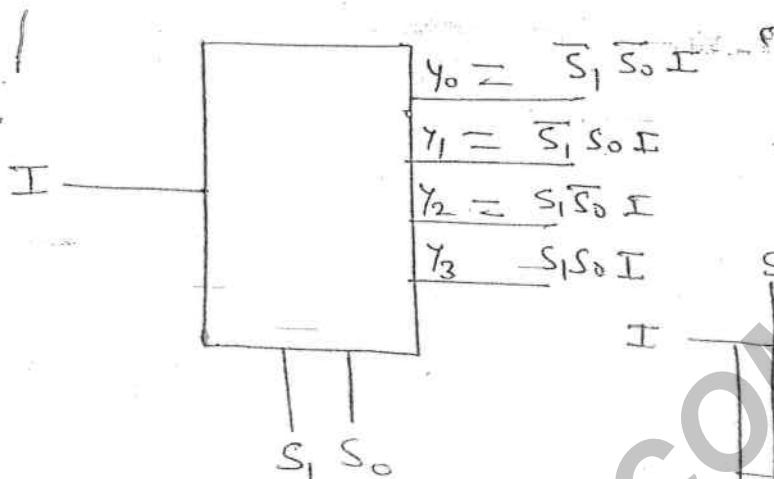
$$Y_0 = \bar{S} \cdot I$$

$$Y_1 = S \cdot I$$

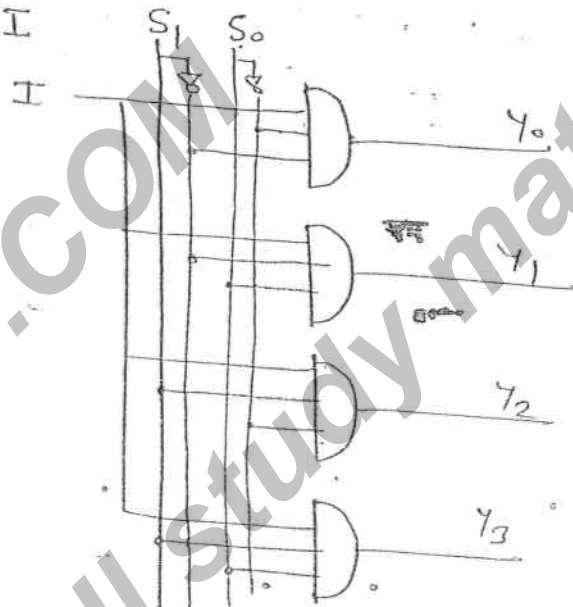


⇒ De mux basically contains only AND gates.

1x4 Demux $\Rightarrow$

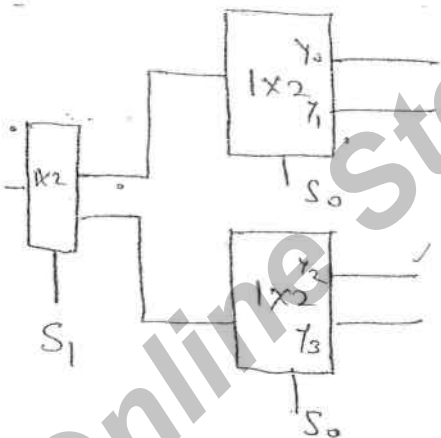


S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0



$\Rightarrow$ Implementation of Higher order Demux Using Lower order Demux

(1) $1 \times 4 \xrightarrow{3} 1 \times 2$



ex: $S_1 S_0 = 10$

$1 \times 8 \xrightarrow{7} 1 \times 2$

$1 \times 16 \xrightarrow{5} 1 \times 4$

$1 \times 64 \xrightarrow{9} 1 \times 8$

$1 \times 256 \xrightarrow{17} 1 \times 16$

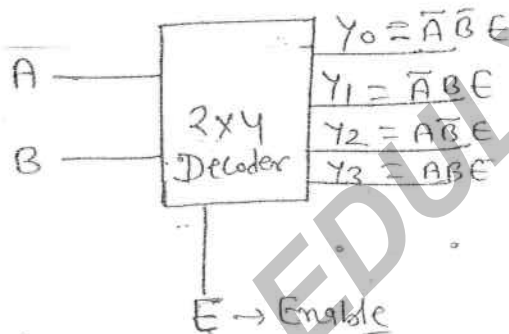
Decoder $\Rightarrow$

43

$\rightarrow$ Decoder is a Combinational ckt which have many inputs and outputs.

$\rightarrow$ it is used to convert binary to other codes such as binary to octal, BCD to decimal, Binary to Hexadecimal etc.

2x4 Decoder $\Rightarrow$



E	A	B	Y_3	Y_2	Y_1	Y_0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

$\rightarrow$ Decoder and Demux Internal ckt. Remain same.

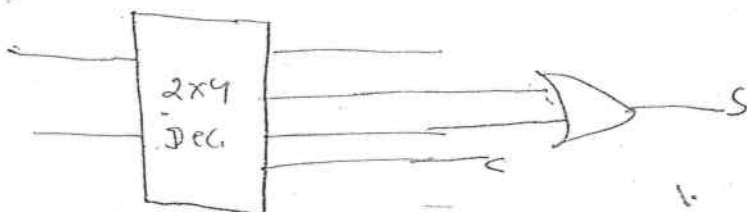
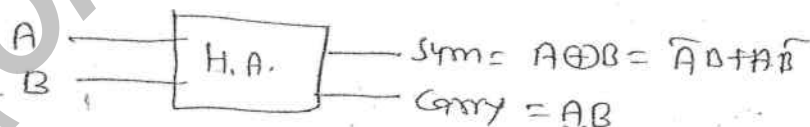
$\rightarrow$ 1×4 Demux with acts as a 2x4 Decoder.

$\rightarrow$ 1×8 Demux $\rightarrow$ 3x8 Decoder

$\rightarrow$ 1×16 Demux $\rightarrow$ 4x16 Decoder

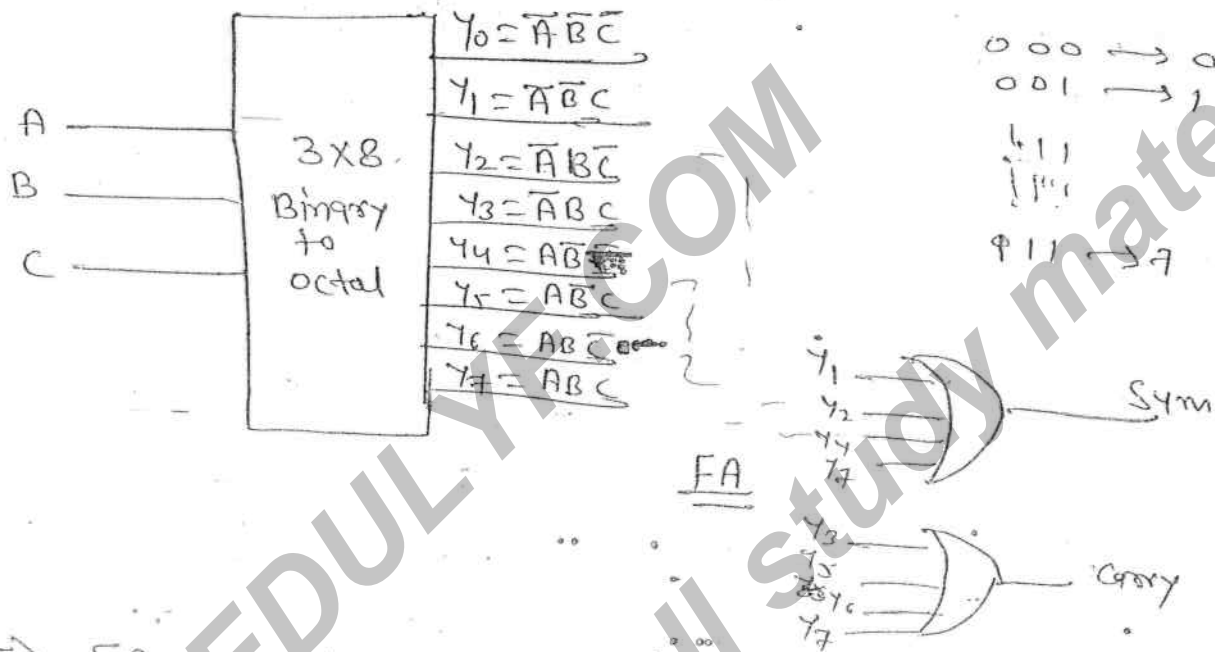
$\rightarrow$ 1×256 Demux $\rightarrow$ 8x256 Decoder

$\Rightarrow$ Implement HA using 2x4 Decoder ckt $\Rightarrow$



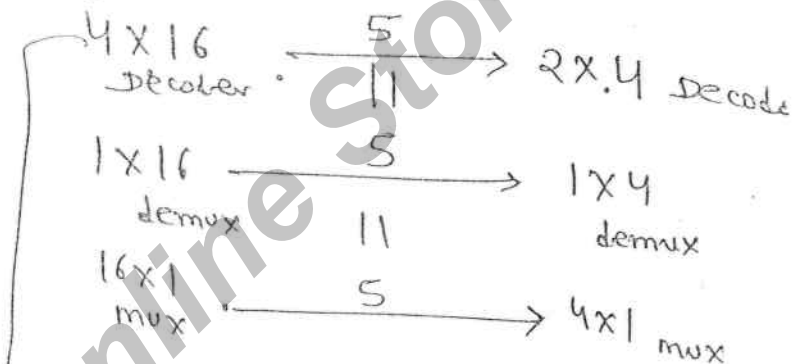
To implement HA (45) ^{OR} one 2x4 decoder
 / one OR gate is required.

3 to 8 Decoder:

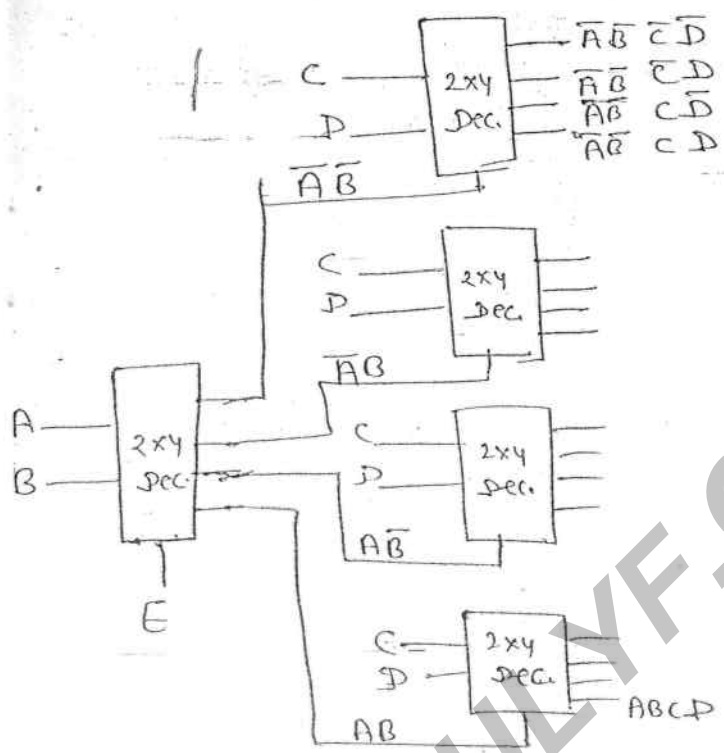


⇒ FA → one 3x8 Decoder and Two 4-input OR gate.

Implementation of Higher order decoder using lower order decoder.



hh

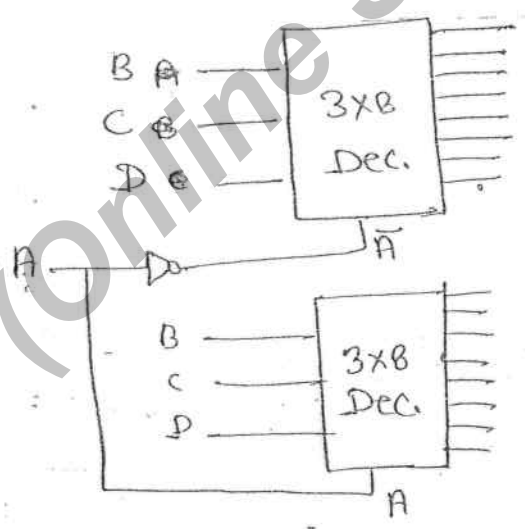


6×64 Dec. $\xrightarrow{21}$ 2×4 Dec.
 1×64 $\xrightarrow{21}$ 1×4 demux
 64×1 $\xrightarrow[16+4+1]{21}$ 4×1

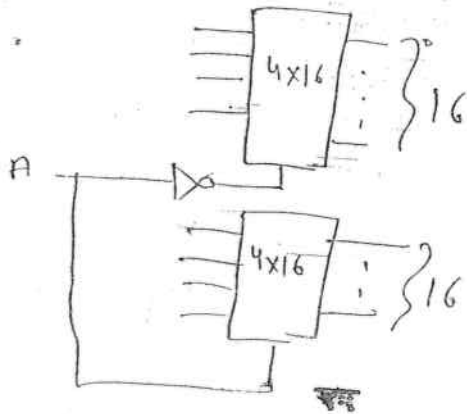
Page 12
 Date

4×16 Dec. $\rightarrow$ 3×8 Dec.

$\hookrightarrow$ Two 3×8 Dec. and one NOT gate



Ex $\Rightarrow$ 5×32 Dec. $\rightarrow$ 4×16 Dec.

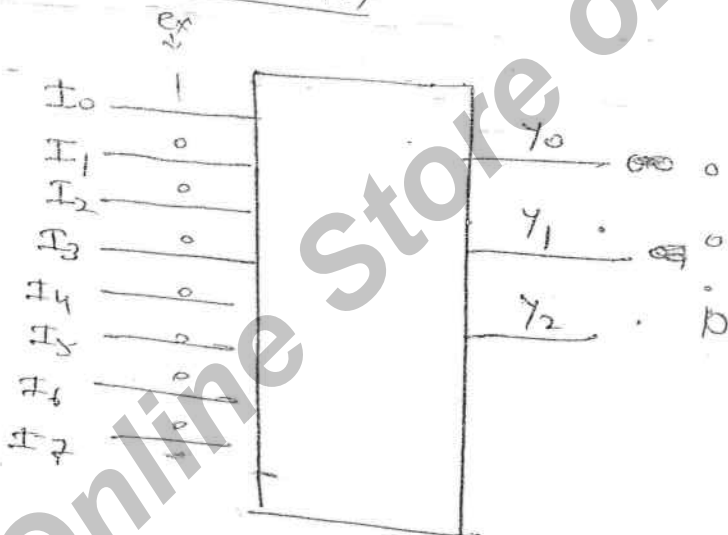


$\Rightarrow$ ENCODER $\Rightarrow$

$\rightarrow$ Encoder is a Combinational ckt which is used to convert other codes to Binary such as.

- Octal to Binary (8×3)
- Decimal to BCD (10×4)
- Hex to Binary (16×4)

Octal to Binary



In Encoder, one of the input line is high and, corresponding binary is available at the output.

I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_2	Y_1	Y_0	
0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	1	0	0	0	1	1
0	0	0	0	0	1	0	0	0	1	0	2
0	0	0	0	1	0	0	0	0	1	1	3
0	0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	0	0	0	0	1	0	1	5
0	1	0	0	0	0	0	0	1	1	0	6
1	0	0	0	0	0	0	0	1	1	1	7

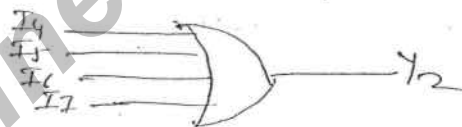
$$\Rightarrow Y_0 = I_1 + I_3 + I_5 + I_7$$

$$Y_1 = I_2 + I_3 + I_6 + I_7$$

$$Y_2 = I_4 + I_5 + I_6 + I_7$$



$\Rightarrow$ Encoder contains OR gates.



Priority Encoder

In priority Encoder any number of inputs can be High based on input Highest priority corresponding binary is available at the output.

2x 4x2 priority

I_3	I_2	I_1	I_0	Y_1	Y_0
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	x	x	x	1	1

Sequential circuits

Flip-flop

- FF is a basic memory element.
- A FF can store one bit.
- FF have two stable-state hence it is known as Bistable multivibrator.
- FF have two outputs which are complementary to each other.

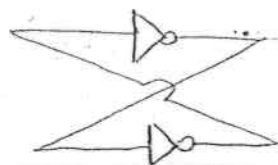
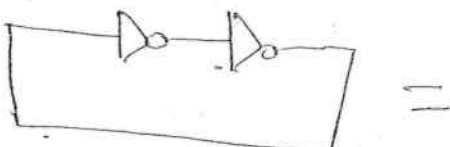
FF →
Syllabus

SR latch < NAND gates
NOR gates

→ SR FF
JK FF
D FF
T FF

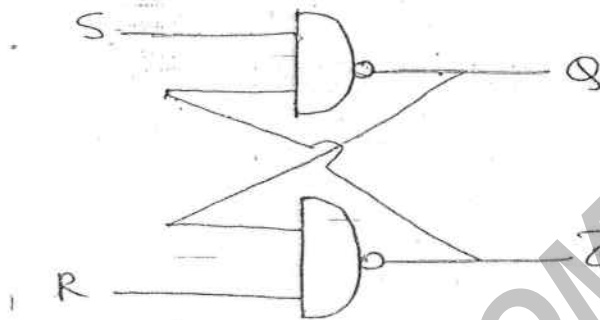
CKT
Truth Table
Characteristic Table
Excitation table → Characteristic equation

- Conversion from one ff to other
- M-S ff



Latch

⇒ SR Latch Using NAND Gate Circuit: ⇒ 46

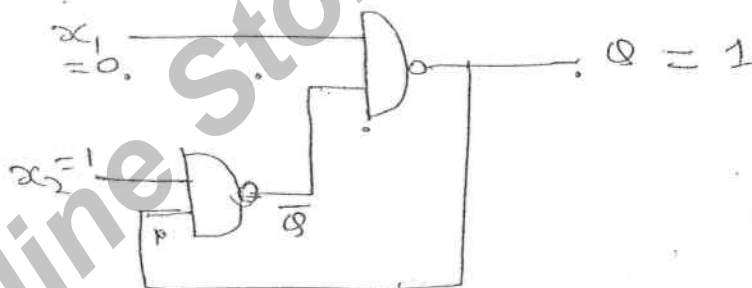


S	R	Q
0	0	Invalid ($Q = \bar{Q} = 1$)
0	1	1
1	0	0
1	1	Previous State ($Q=0; \bar{Q}=1$) (OR) ($Q=1; \bar{Q}=0$) *

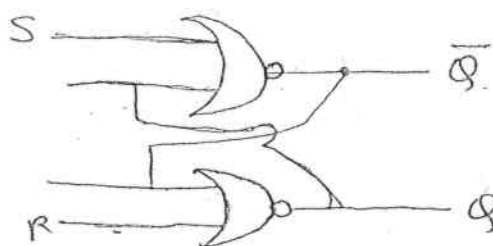
$E=1$
$d=0$

NAND		
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

Ques → In the ckt shown in fig. if $x_1=0$ & $x_2=1$ then output Q will be



⇒ SR Latch Using NOR Gate: ⇒



NOR		
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

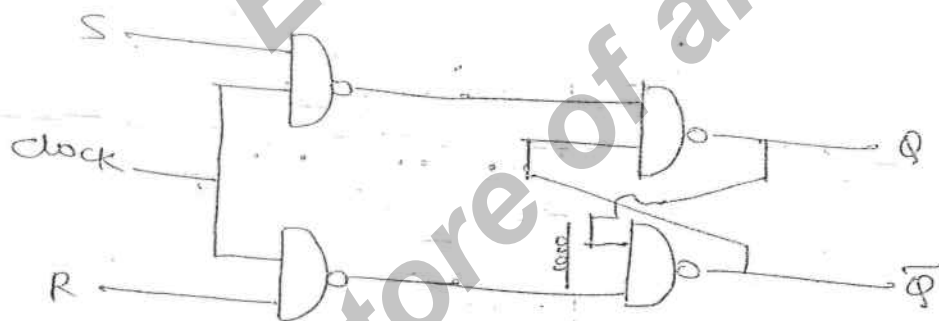
S	R	Q
0	0	previous-state (both inputs are enable)
0	1	0
1	0	1
1	1	Invalid ($Q=0, \bar{Q}=0$)

NOR Latch

- SR Latch is used as basic memory element in all flip-flops.
- it is used to eliminate bounce in key-board or switches.



SR FF



Clock	S	R	Q_{n+1}
0	X	X	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	Invalid

Hold state

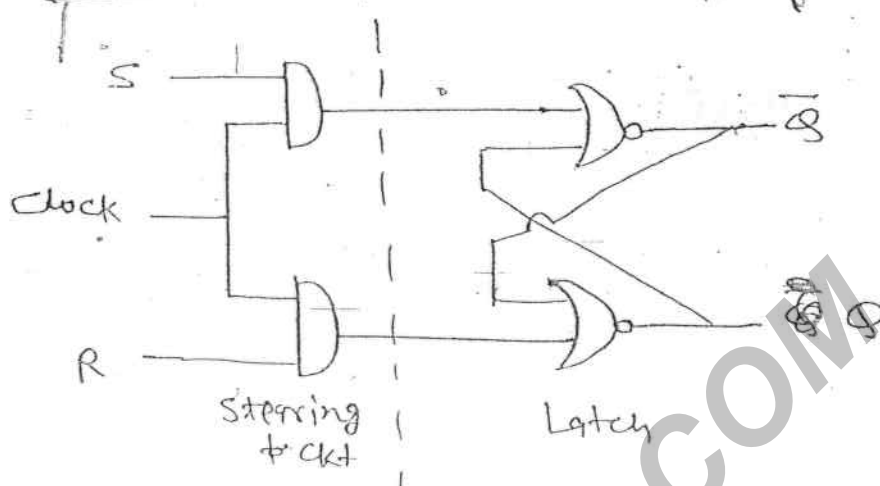
Reset state

Set state

Unused state

(NOR Latch)

⇒ SR FF using NOR Latch: →



Clock	S	R	Q_{n+1}
0	X	X	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	Invalid

← Same as NAND SR using NAND latch

SR FF Characteristic Table: →

	S	R	Q_n	Q_{n+1}
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1
5	1	0	1	1
6	1	1	0	X
7	1	1	1	X

$$Q_{n+1} = \sum m(1, 4, 5) + \sum d(6, 7)$$

	RQ_n	00	01	11	10
S	0		1		
	1	1	1	d	d

$$Q_{n+1} = S + \bar{R}Q_n$$

This ^{eqn} is valid only when $S \cdot R = 0$

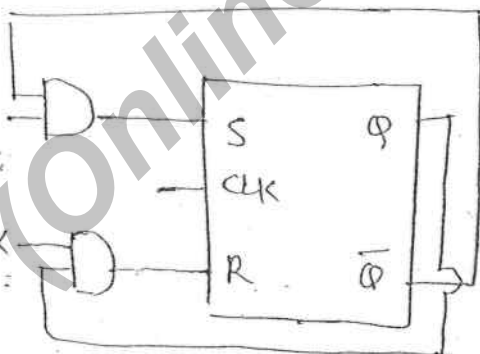
Excitation Table $\Rightarrow$

Q_n	Q_{n+1}	S	R
0	0	0	d
0	1	1	0
1	0	0	1
1	1	d	0

$\rightarrow$ Disadvantage of SR flip-flop is invalid state with present when both S and R inputs are 1. to avoid this JK flip-flop is used.

JK Flip flop

01/09/2011



$$S = J \bar{Q}$$

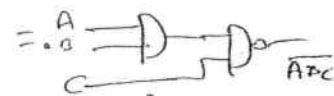
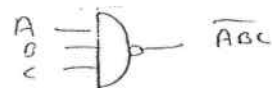
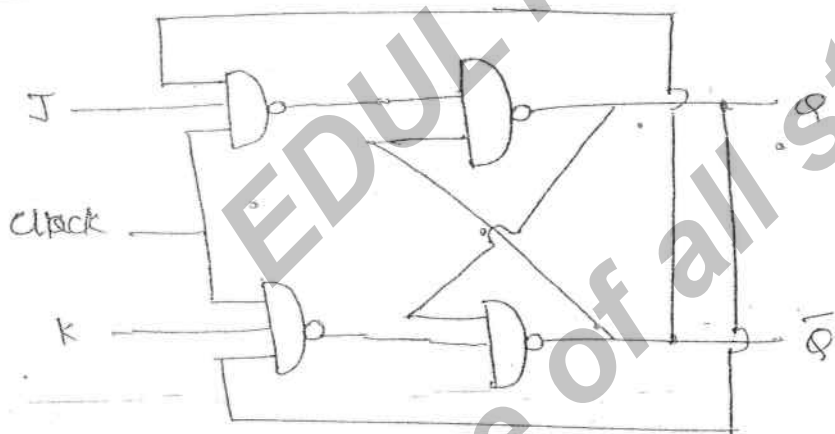
$$R = KQ$$

J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	$\bar{Q}_n$

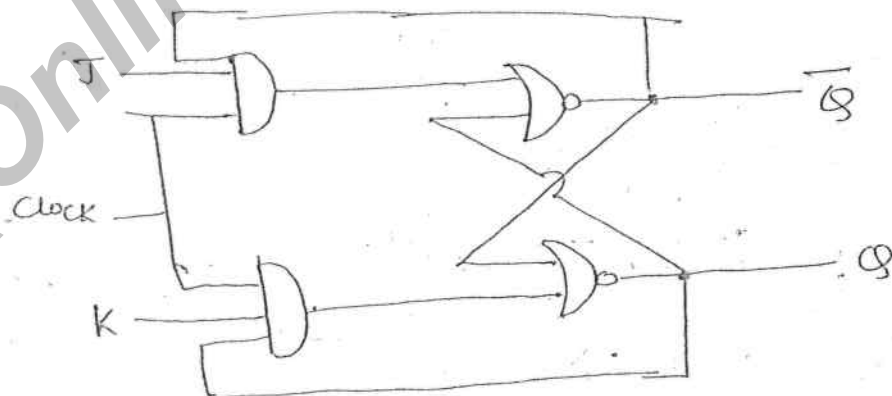
Truth Table

Clock	J	K	Q_{n+1}
0	X	X	Q_n
1	0	0	Q_n ← Hold state
1	0	1	0 ← Reset state
1	1	0	1 ← Set state
1	1	1	$\bar{Q}_n$ ← Toggle state

⇒ JK Flip-flop Using NAND Latch :-



⇒ JK Flip-flop Using NOR Latch :-



⇒ Characteristic Table of JK ff

J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Characteristic eqn ⇒

$J \backslash K Q_n$	00	01	11	10
0	0	1	1	0
1	1	1	1	1

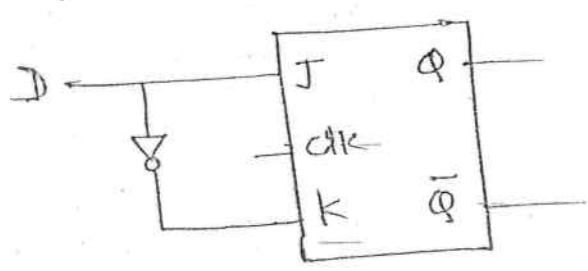
⇒

$$Q_{n+1} = J \bar{Q}_n + \bar{K} Q_n$$

Excitation Table

Q_n	Q_{n+1}	J	K
0	0	0	d
0	1	1	d
1	0	d	1
1	1	d	0

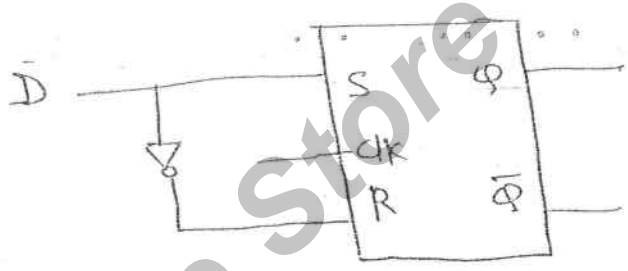
D. Flip-flop $\Rightarrow$



$$J = D, K = \bar{D}$$

Truth Table

clk	D	Q_{n+1}
0	X	Q_n
1	0	0
1	1	1



$$S = D; R = \bar{D}$$

Characteristic Table

D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

$$Q_{n+1} = D\bar{Q}_n + DQ_n = D(Q_n + \bar{Q}_n)$$

$$Q_{n+1} = D$$

$\Rightarrow$

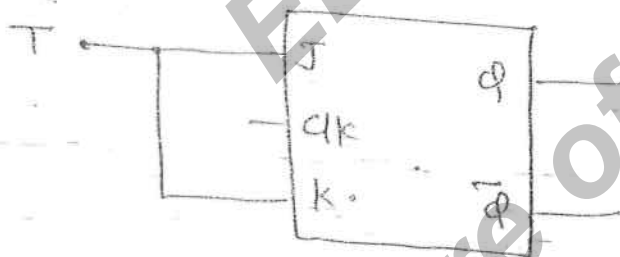
⇒ D flip-flop is known as Transparent Latch.

Excitation Table

Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

D → Delay (or) Data

T flip-flop



$$J = K = T$$

Truth Table

clk	T	Q_{n+1}
0	X	Q_n
1	0	Q_n ← HOLD Hold
1	1	$\bar{Q}_n$ ← Toggle

Character

T	Q_{n+1}
0	Q_n
1	$\overline{Q_n}$

50

Characteristic Table: $\Rightarrow$

T	Q_n	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

$$\Rightarrow Q_{n+1} = \overline{T}Q_n + T\overline{Q_n}$$

$$Q_{n+1} = T \oplus Q_n$$

Excitation Table: -

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	$\overline{Q_n}$

SR

DDF

T FF

$$Q_{n+1} = S + \bar{R} Q_n$$

$$Q_{n+1} = J \bar{Q}_n + \bar{K} Q_n$$

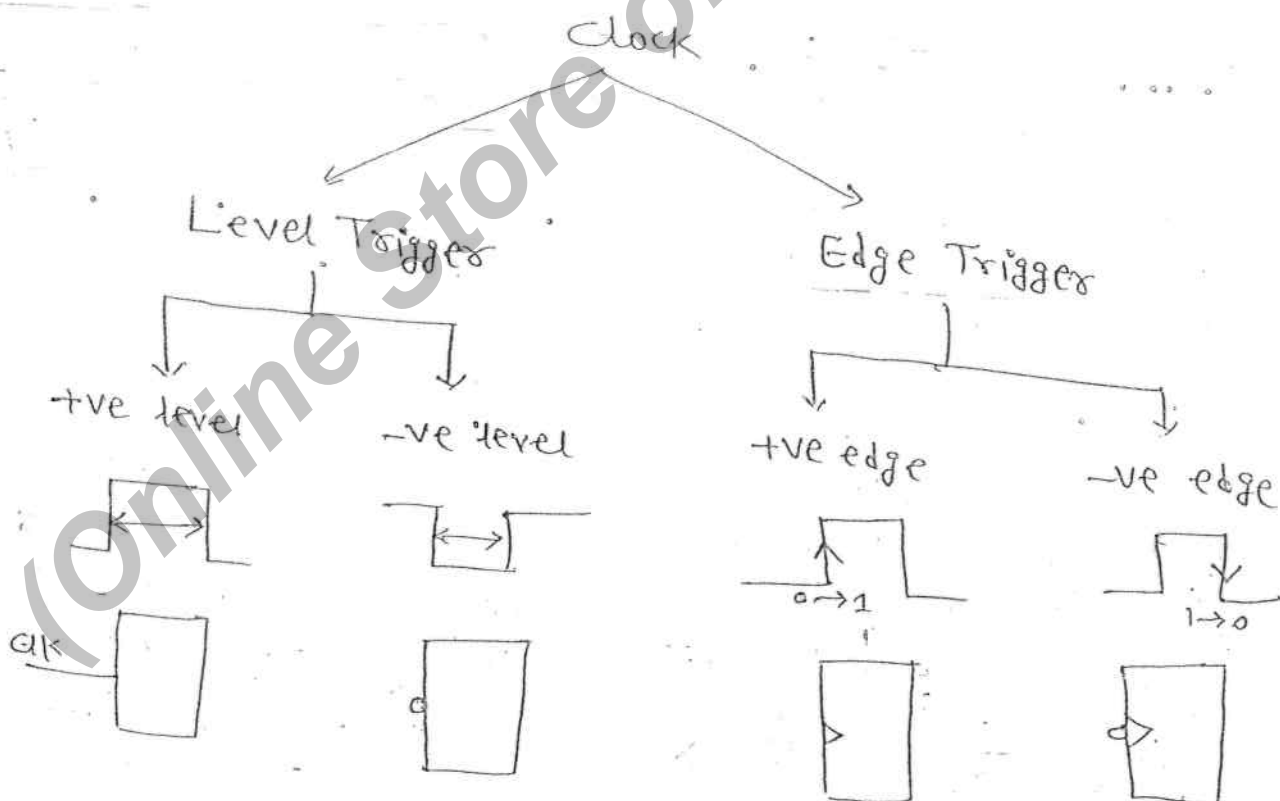
$$Q_{n+1} = D$$

$$Q_{n+1} = T \oplus Q_n$$

Excitation Table

Q_n	Q_{n+1}	S	R	J	K	D	$\bar{T}$
0	0	0	X	0	X	0	0
0	1	1	0	1	X	1	1
1	0	0	1	X	1	0	1
1	1	X	0	X	0	1	0

$JK \rightarrow$ Jack Kilby



→ In edge Triggering output will change only once in a single clock.

→ In level trigger circuit output may change many times in single clk pulse if input is changing.

⇒ RACE-AROUND Condition

⇒ Disadvantage of JK FF is Race around condition

→ In JK FF Race Around occurs when $J = K = 1$ and propagation delay of Flip-flop is less than the pulse width of clock.

$$\Rightarrow \begin{array}{l} J = K = 1 \\ \text{and } t_{pff} < t_{pw} \end{array}$$

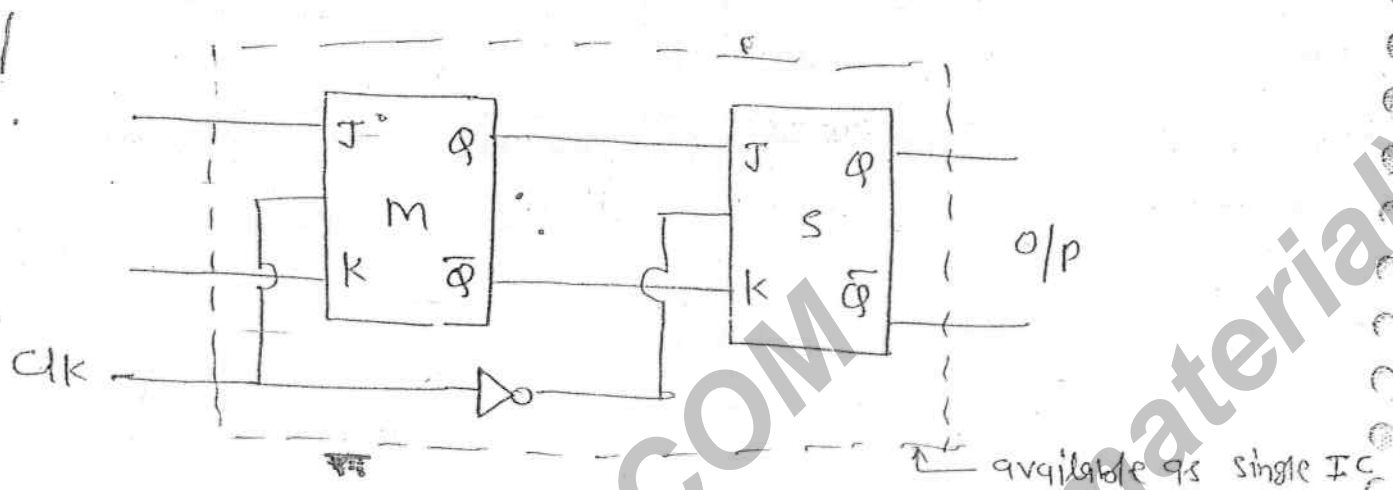
→ During Race Around output will change many times in single clock.

→ Condition to avoid Race Around Condition

$$t_{pw} < t_{pff} < T_{CLK}$$

→ To avoid Race Around Condition master-slave Flip-flop configuration is used.

Master - Slave Flip flop



- The ms configuration shown in fig. master is applied with input clock and slave is applied with inverted clock.
 - in ms configuration output changes only when slave output is changing.
 - There is no Race around at output of slave.
 - Based on operation master is level triggered slave is edge triggered.
 - in above fig. → master is +ve Level triggered and slave is -ve edge triggered.
- ⇒ Conversion from one flip flop to other flip-flop:

- (i) Fill the ~~char~~ Required ff characteristic table.
- (ii) Fill available ff excitation table.
- (iii) Write excitation equations.
- (iv) minimization
- (v) Implement.

JK F

(*)

D	Q_{n+1}	J	K
0	0	0	X
0	0	X	1
1	1	1	X
1	1	X	0

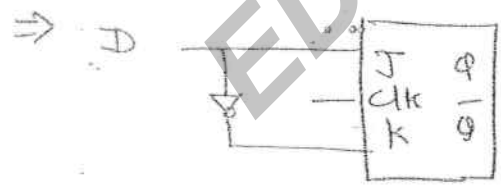
(*)

D	Q _n	0	1
0	0	0	X
1	0	1	X

$J = D$

D	Q _n	0	1
0	0	X	1
1	0	X	0

$K = \bar{D}$



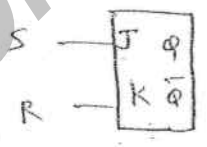
JK to D

$J = D, K = \bar{D}$

JK to T

$J = \bar{T}, K = T$

JK to SR



$J = S, K = R$

T to D

D	Q_n	Q_{n+1}	T
0	0	0	0
0	1	0	0
1	0	1	1
1	1	1	0

D	Q_n	
0	0	0
0	1	0
1	0	1
1	1	0

$$\Rightarrow T = D \oplus Q_n = \bar{D}Q_n + D\bar{Q}_n$$

D to T

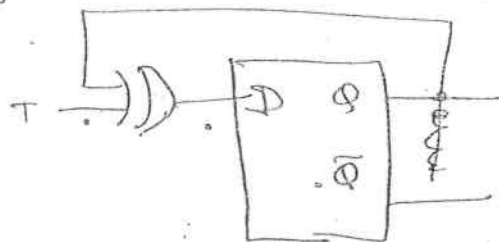
T	Q_n	Q_{n+1}	D
0	0	0	0
0	1	1	0
1	0	1	1
1	1	0	1

$$D = \bar{T}Q_n + T\bar{Q}_n = T \oplus Q_n$$

$$Q_{n+1} = D$$

$$Q_{n+1} = T \oplus Q_n$$

$$D = T \oplus Q_n$$



D to JK

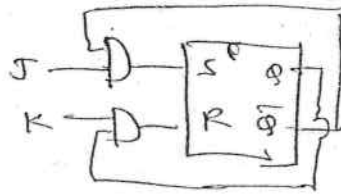
$$D = J\bar{Q} + KQ$$

D to SR

$$D = S + \bar{R}Q$$

SR to JK

$$S = J\bar{Q} ; R = KQ$$



S3

SR to D

$$S = D$$

$$R = \bar{D}$$

SR to T

$$S = T\bar{Q}$$

$$R = TQ$$

T to JK : $\Rightarrow$

$$T = J\bar{Q} + KQ$$

T to SR

$$T = S\bar{Q} + RQ$$

T to D

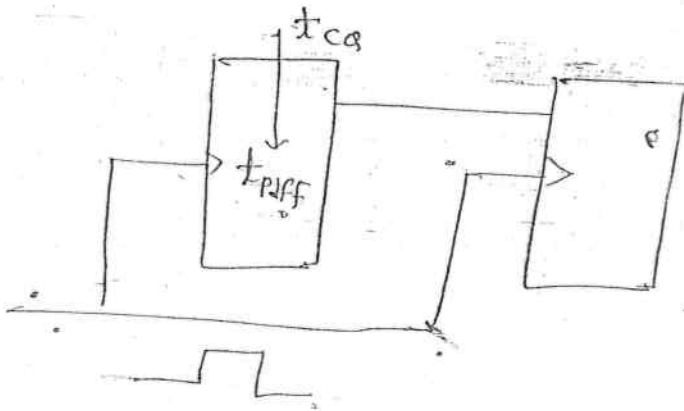
$$T = D \oplus Q$$

Set-up time is the minimum time required to keep input at proper level before applying clock.

Hold Time is the minimum time required to keep same data after applying clock to store properly.

If Set up time and Hold time not given properly then output will go to meta stable state.

(i)

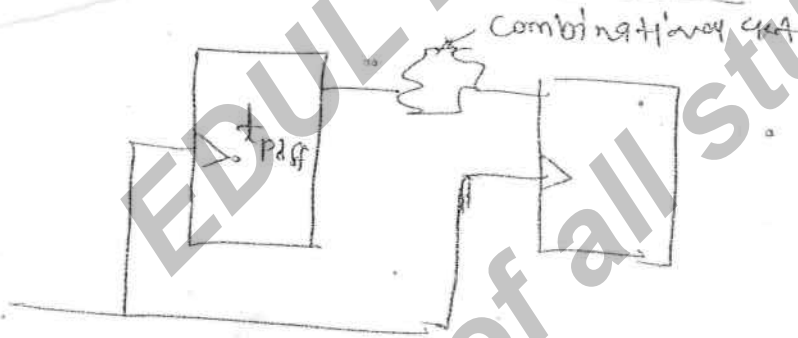


$T_{clk} \rightarrow$ Time period of clock

$$T_{clk} > t_{pdff} + t_s$$

t_{pdff} (OR) $t_{cq} \rightarrow$ clock to queue delay,
 $t_s \rightarrow$ setup time

(ii)



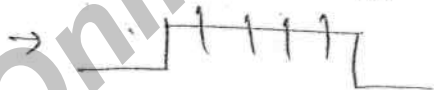
$$T_{clk} > t_{pdff} + t_{comb} + t_s$$

Latch

Ff

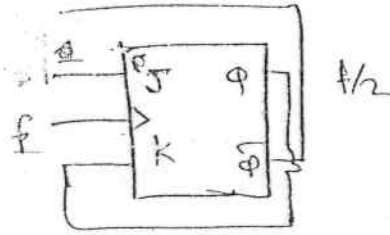
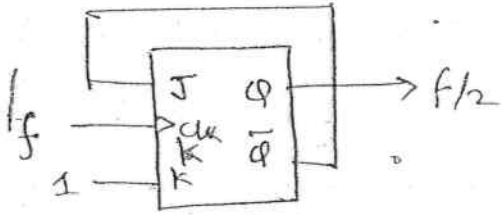
$\rightarrow$ Level triggered

Edge triggered

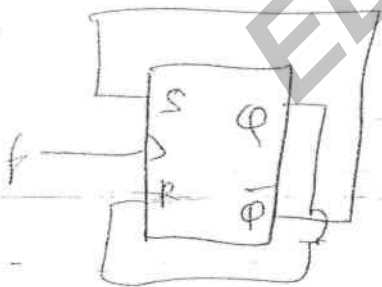
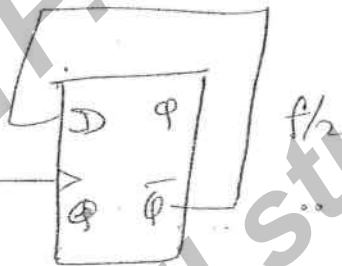
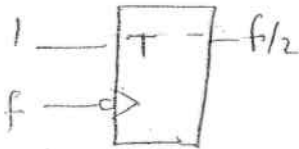
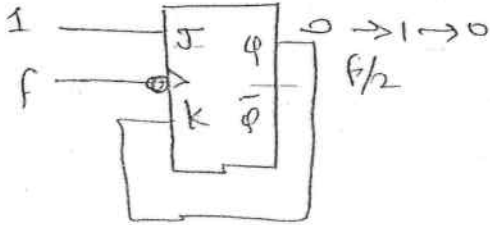


asynchronous

synchronous



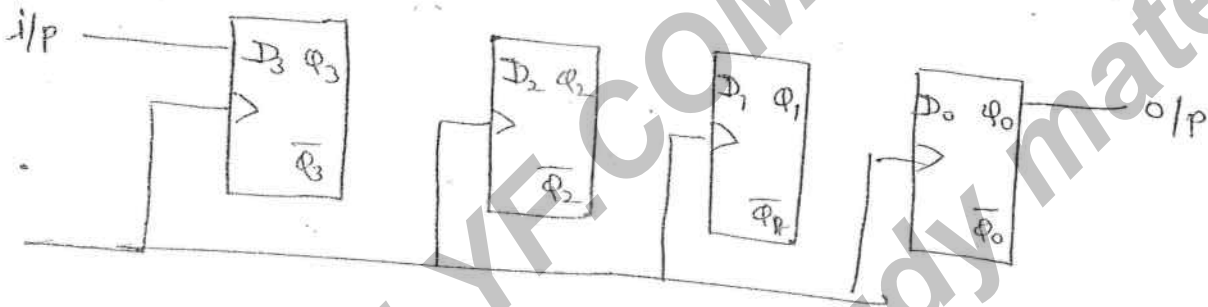
54



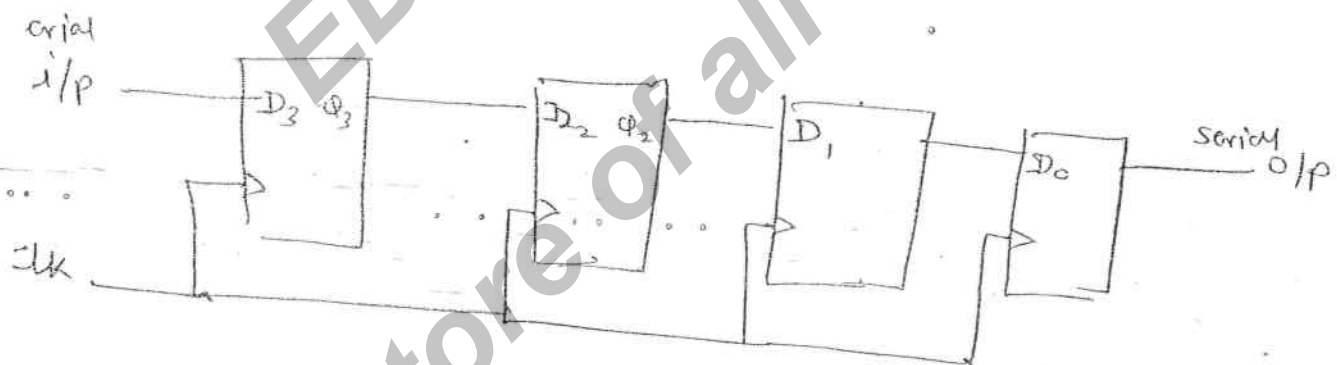
Toggle (7)

Registers

- Registers are used to store group of bits.
- in registers to store n bits n flip flops are used.
- Depending on i/p and o/p registers can be classified into 4 types → SISO, SIPO, & PISO, PIPO.



SISO



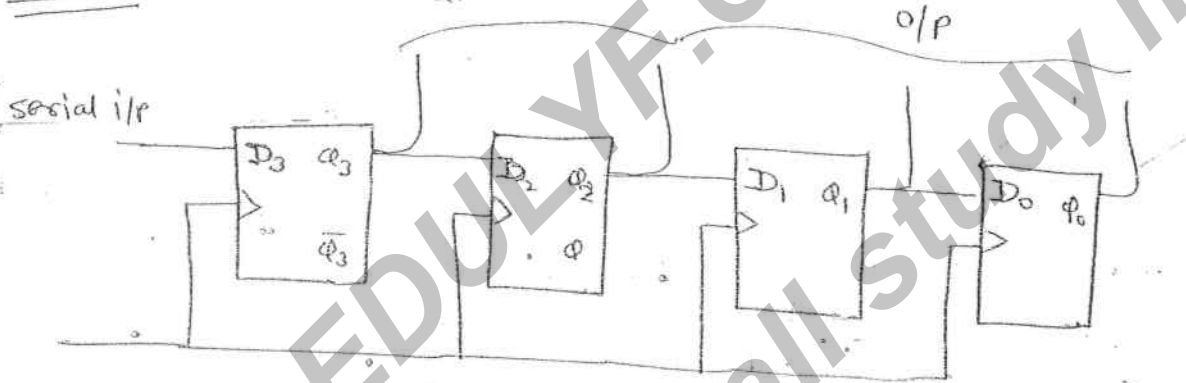
Clock	Data	Q_3	Q_2	Q_1	Q_0
0	1011	0	0	0	0
1		1	0	0	0
2		1	1	0	0
3		0	1	1	0
4		1	0	1	1

→ n bit SISO Register to store n -bit data —
it require n clock pulse.

→ SISO Register will provide n clock pulse delay to the input data. (delay = nT_{clk}).

→ To provide serial output data it required minimum $(n-1)$ clock pulses

SISO Register ** $\leftarrow \overline{V} \overline{V} \text{Imp}$



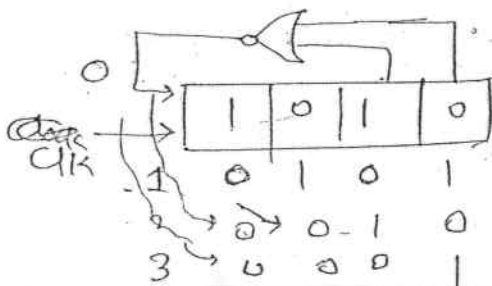
SISO → Serial i/p → n clock pulse

parallel o/p → 0 clock pulse (minimum)

~~minimum no. of~~

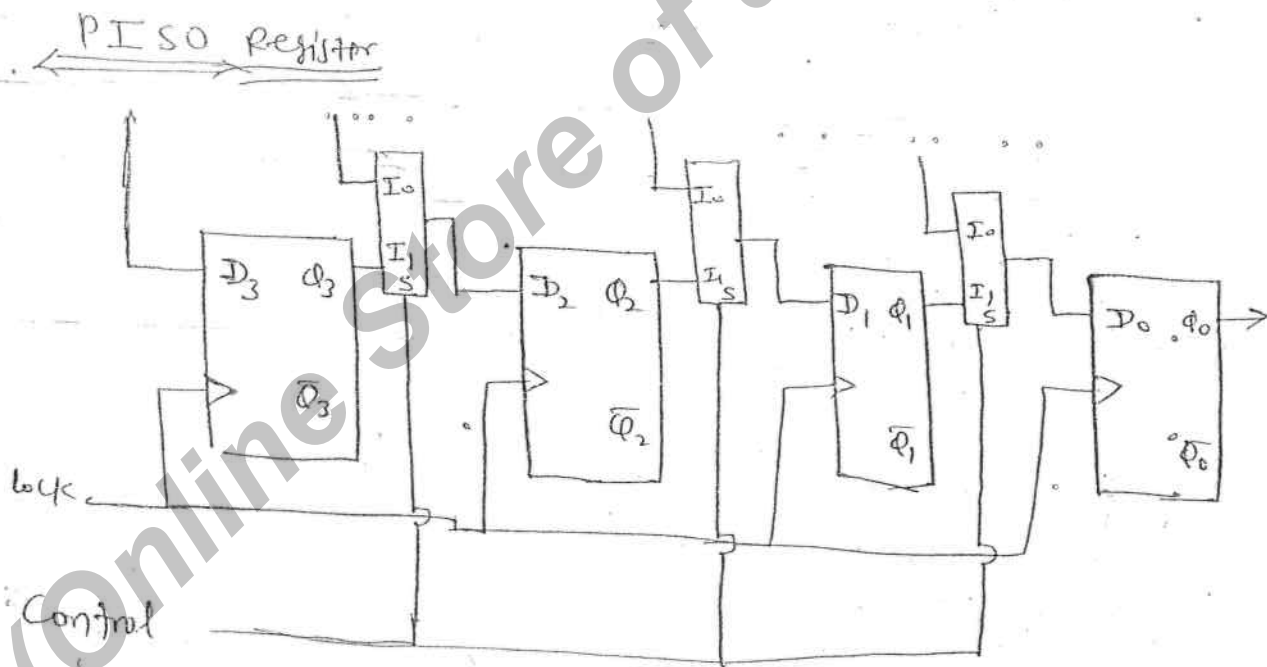
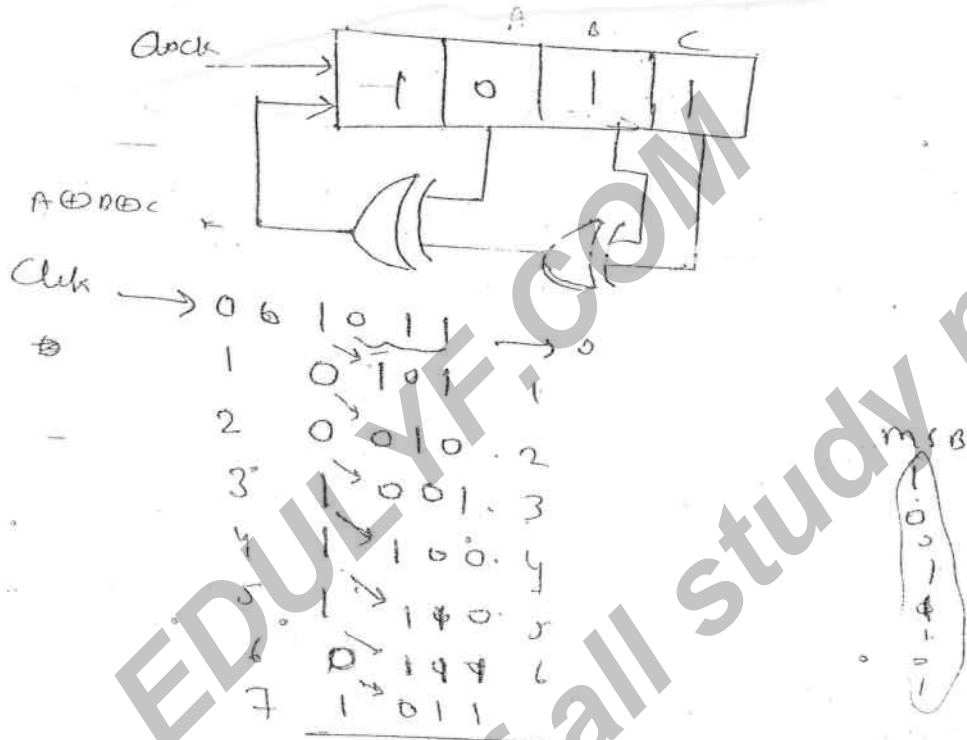
Serial → Parallel
↓ ↓
Temporal code. Special code.

Q. The ckt shown in fig is a 4-bit SISO Register which is initially loaded with 1010, after 3-clock pulses The content of Register is



Any →

Q. The ckt shown in fig. is 4-bit SIPO Register loaded with 1011 if clock pulses are applied continuously then after how many pulses again data will become 1011 again.

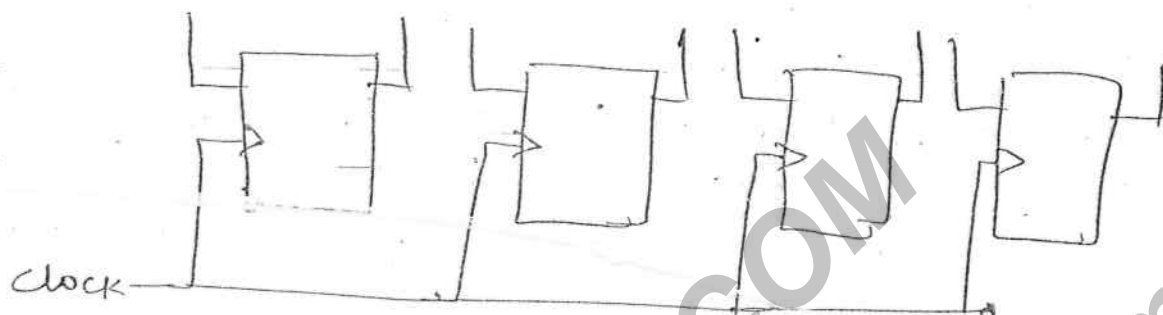


if Control = 0 → PI
Control = 1 → Serial o/p

Converts
 → Special code → ~~to~~ ^{to} Temporary code.

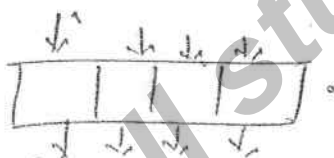
56

PIPO



for Parallel i/p → 1 clock pulse

for - program o/p → 0 clock pulse



→ No shift operation required.

→ PIPO used as storage or ~~storage~~ register in microprocessor
 → it will not provide shift operations.

SI	i/p	o/p
SI SO	n	n-1
SI PO	n	0
PI SO	1	n-1
PI PO	1	0

Clock pulse

Shift Register

→ in Shift left Register each shift left operation will provide multiply by 2.
 → if n shift ^{left} operation will be ~~perman~~ performed then data will be multiplied by 2^n .

→ if n shift R operation with one performed then data will divide by 2^n .

Counters : →

- Counters are basically used to count No. of clock pulses applied.
- Counter can also be used as → frequency divider,
 - Timers, w
 - wave form generation
 - frequency measurement
 - Distance / range measurement
 - pulse width measurement.
- In counters with n flip-flops maximum possible states are 2^n .

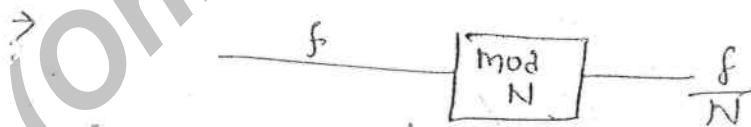
→ Let N is No. of states, n is No. of flip-flops.

then

$$N \leq 2^n$$

$$\Rightarrow n \geq \log_2 N$$

→ No. of states for used in a counter known as modulus No. or mod ~~count~~ No. Number.

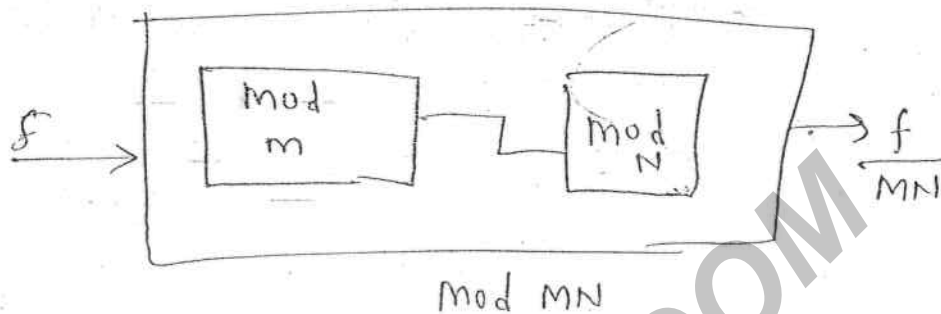


→ A decade counter is applied with a clock freq. of 10 MHz then output freq. is.

Ans

$$\frac{10 \text{ MHz}}{10} = 1 \text{ MHz}$$

→ if mod m counter & mod N counter is cascaded then it acts as mod MN counter. 57



→ Depending on clock pulses counter can be identified as Synchronous & asynchronous counter.

Synchronous	Asynchronous
→ all ff applied with same clock	→ in this different flip-flops are applied with different clock pulses
→ Faster	→ slower
→ Any count sequence is possible with Synchronous counter.	→ fixed count sequence is present. it either up OR Down
→ No Decoding errors will present in this counter	→ Decoding errors will present
Ex: Ring counter, Johnson counter	Ripple counter

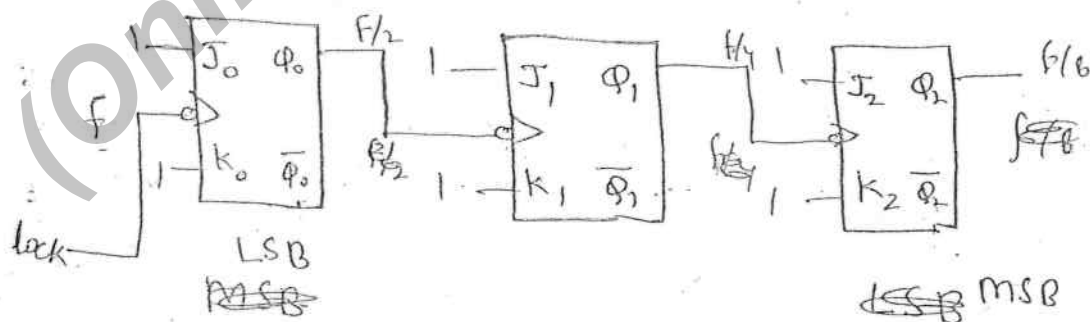
Basics

- (2) Ripple Counter *
- (3) BCD Counter, mod 5
- (4) Ring Counter
- (5) Johnson Counter
- (6) Synchronous Series
- (7) Synchronous Parallel
- (8) Synchronous Counter Design

Ripple Counter

- Ripple Counter is asynchronous counter.
- In Ripple Counter flip-flops are operated in Toggle mode.
- In this different flipflops are applied with different clock frequency.
- In Ripple Counter If clock is applied to only one flip flop and output of one flip flop with acts as clock to the next flip flop.
- In Ripple Counter the flip-flop applied with external clock will acts as LSB bit.

3-bit Ripple Counter : $\Rightarrow$ (up counter)



→ -ve edge trigger $\Rightarrow$ clock $\Rightarrow$ flip flop

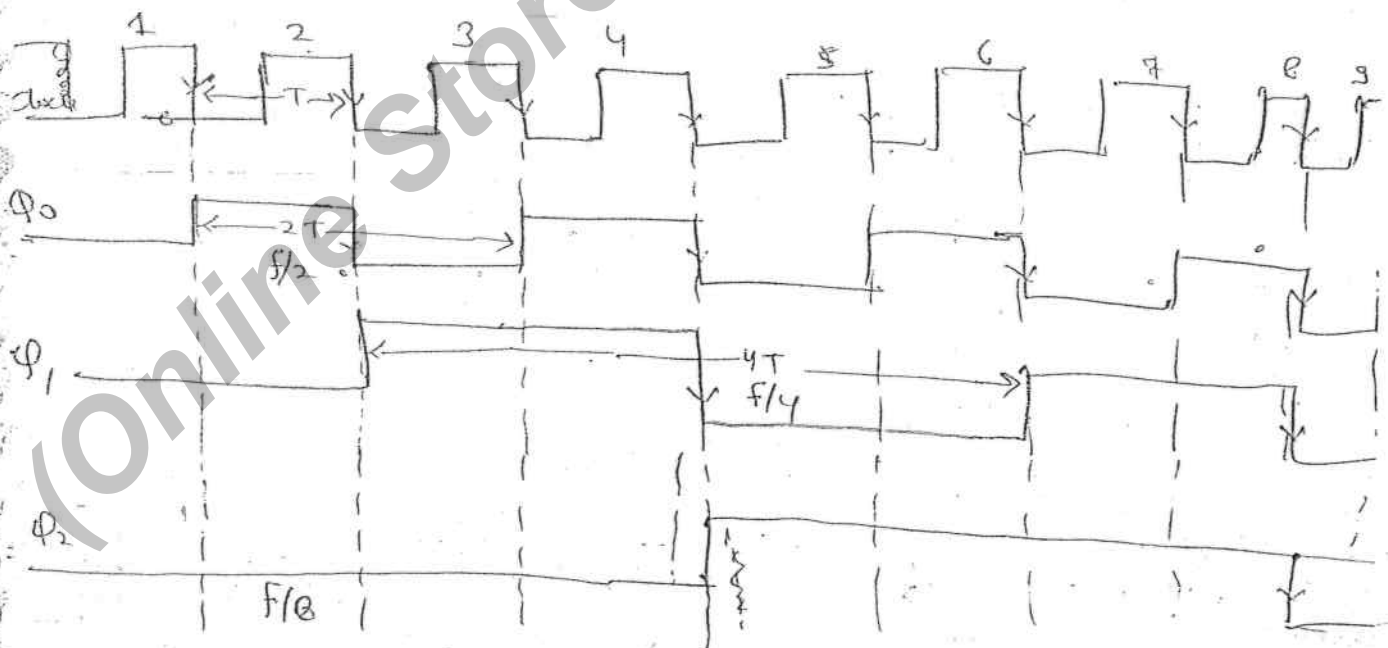
→ In the ckt shown in fig. Φ_0 Toggles for every clock pulse -ve edge. 58

→ Φ_1 Toggles when Φ_0 is changing from 1 to 0.

Φ_2 Toggles when Φ_1 is changing from 1 to 0.

Clock	Φ_2	Φ_1	Φ_0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8	0	0	0

B-state



→ with n flip-flops maximum possible state is 2^n .

→ if f is clock frequency the output frequency is $f/2^n$ Hence Ripple Counter is also known as $2^n:1$ scalar, divider.

→ In a Ripple Counter each flip-flop has propagation delay of t_{pdff} . Then Time period of clock (T_{clk}) is

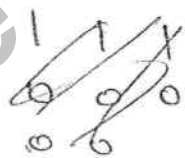
$$T_{clk} \geq n t_{pdff}$$

or

$$f_{clk} \leq \frac{1}{n t_{pdff}}$$

$$f_{max} = \frac{1}{n t_{pdff}}$$

→ Transient states or decoding error will present in ripple counter due to propagation delay.



1	1	1
1	1	0
1	0	0
0	0	0

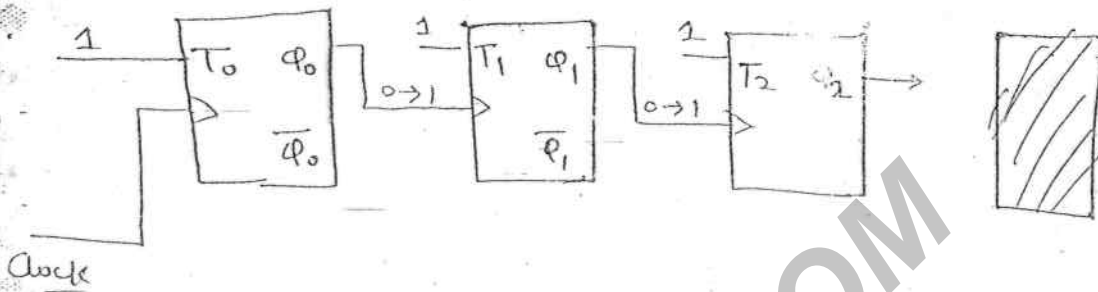
} → Unwanted state

→ In Ripple counter -ve edge ~~with~~ trigger with ϕ as clock will act as up counter.

→ +ve edge triggering with $\bar{\phi}$ as clock is up counter.

Down Counter

58

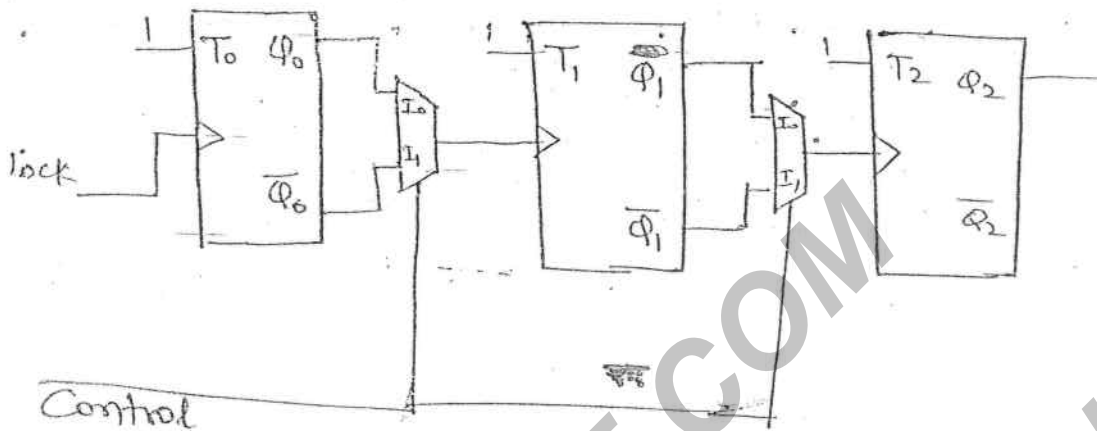


clk	Q ₂	Q ₁	Q ₀
0	0	0	0
1	1	1	1
2	1	1	0
3	1	0	1
4	1	0	0
5	0	1	1
6	0	1	0
7	0	0	1
8	0	0	0

- +ve edge trigger Q as clock down counter
- -ve edge trigger $\bar{Q}$ as clock down counter
- -ve edge trigger $\bar{Q}$ as clock up counter
- +ve edge trigger Q as clock up counter

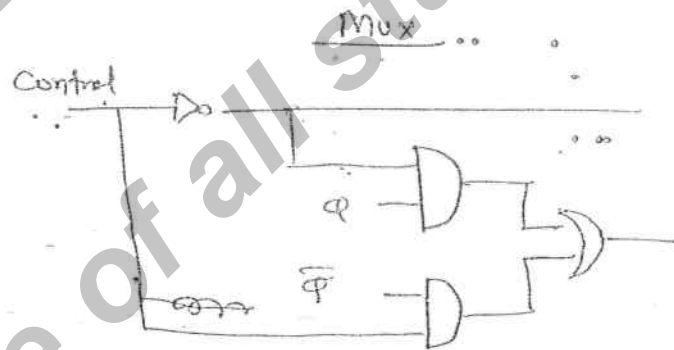
Up/Down counter

02/09/2011

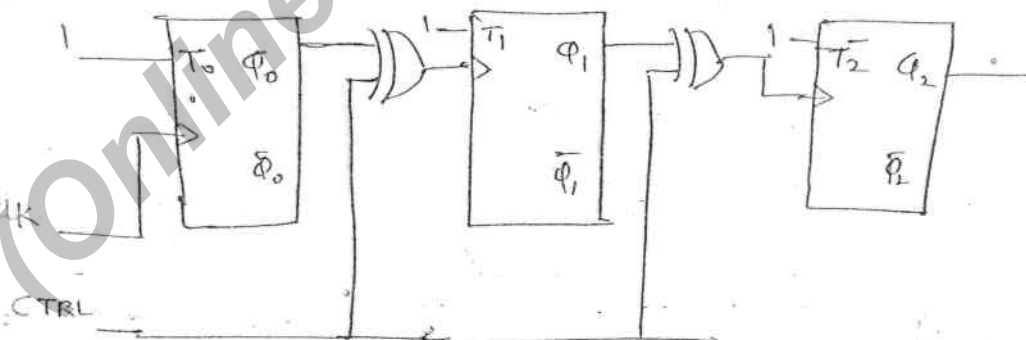
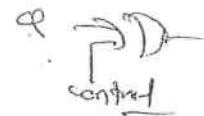


Control = 0 $\rightarrow$ Down counter

Control = 1 $\rightarrow$ Up counter



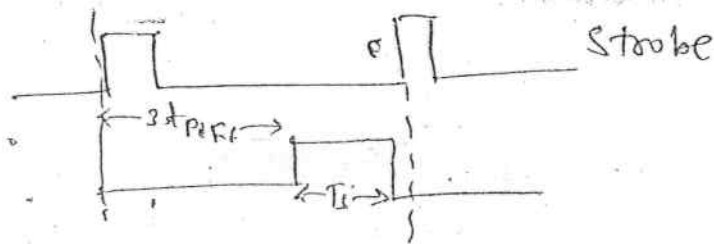
OR mux Replaced by Ex-OR gate



if CTRL = 0 down counter

CTRL = 1 up counter

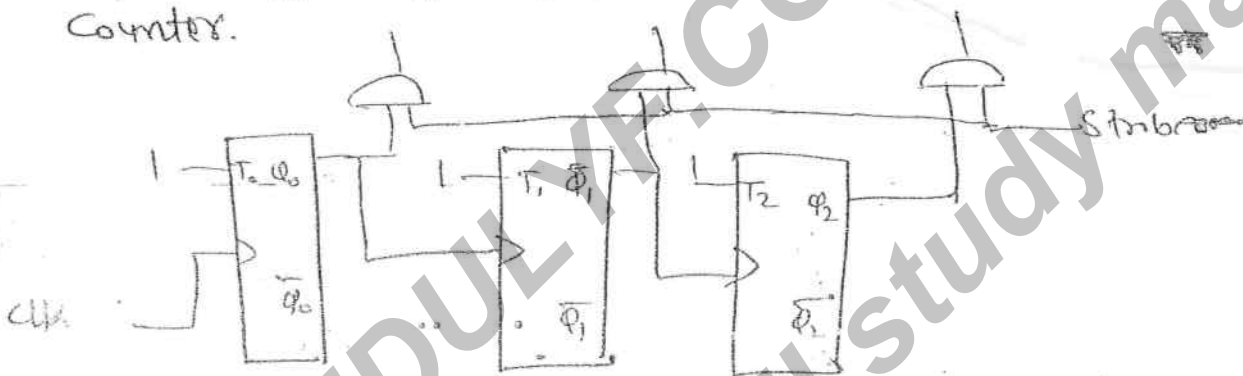
0	0	0
0	0	1
0	1	1
1	1	1



60

$$T_{\text{strobe}} \geq n T_{\text{per}} + T_c$$

Strobe signals are used to avoid decoding error in Ripple Counter.



Clear and presets are used to eliminate some of the states in counter irrespective of clock.

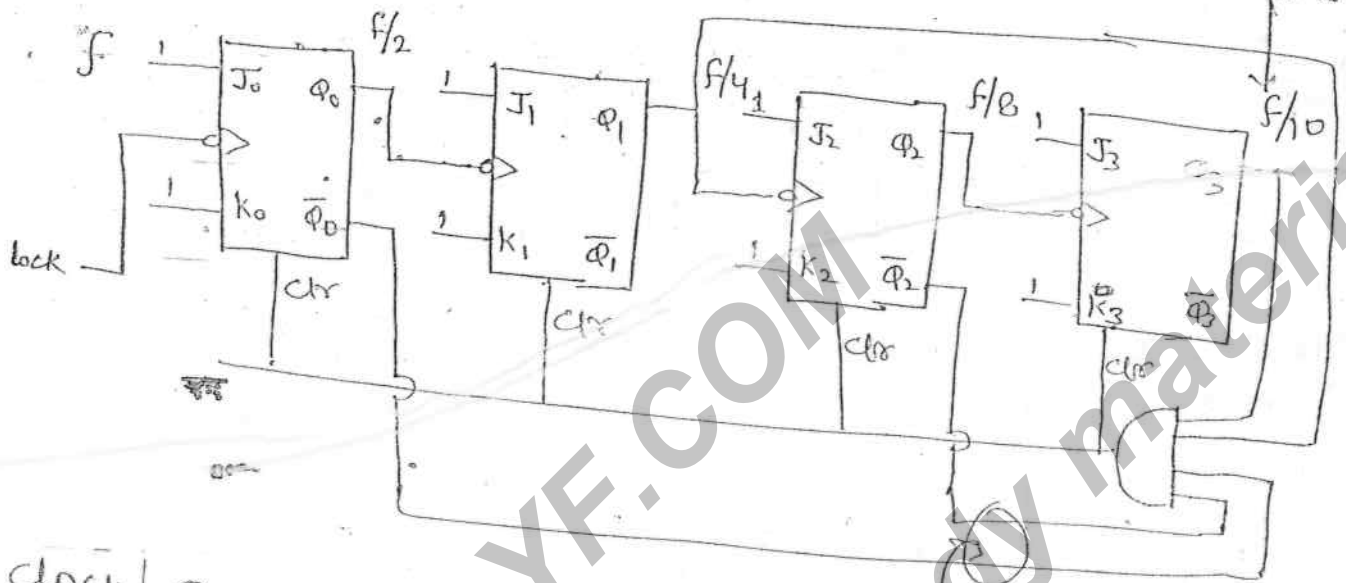
Clear is used to Reset counter to zero.

preset is used to set counter output to ^{logic} High.

$J, K, D, T, S, R \rightarrow$ synchronous input $\therefore$ affects only when clock applied.

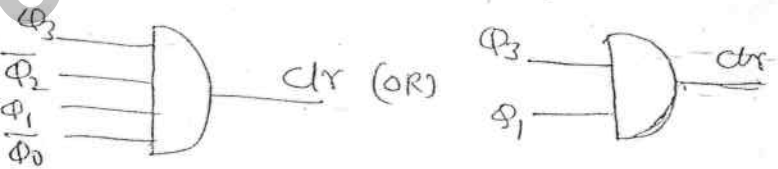
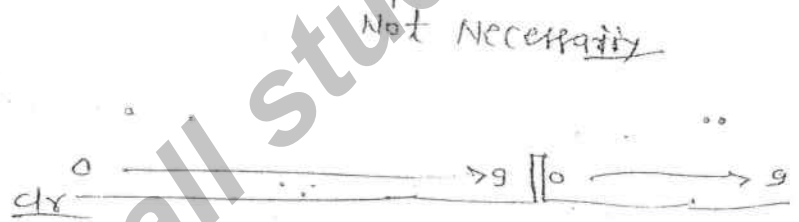
clear, preset $\rightarrow$ asynchronous

BCD Counter $\Rightarrow$ 4 FF's
10 states 0 to 9



clock	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	0	0	0	0

$Q_3 \bar{Q}_2 \bar{Q}_1 \bar{Q}_0$



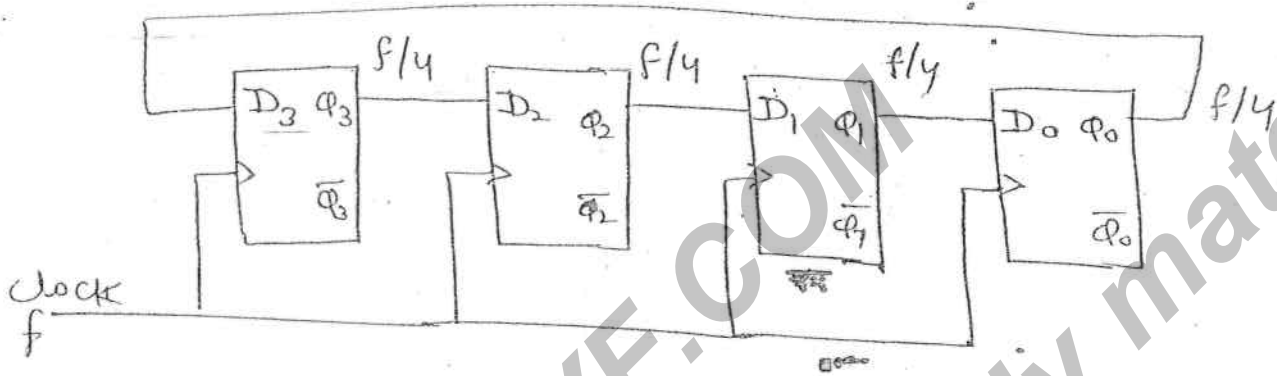
	Q_3	Q_2	Q_1	Q_0
10	1	0	0	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

$Q_3 \bar{Q}_1$

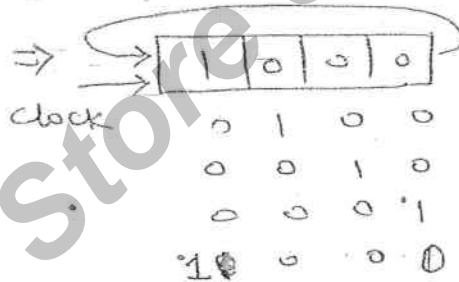
Synchronous Counter : $\Rightarrow$

61

Ring Counter : $\Rightarrow$



- $\rightarrow$ Ring counter is a shift counter in which feedback is applied.
- $\rightarrow$ In this counter only one flip-flop output is logic 1 and remaining flip-flops are logic 0.
- $\rightarrow$ When clock pulse is applied logic 1 is propagate among all flip-flops.

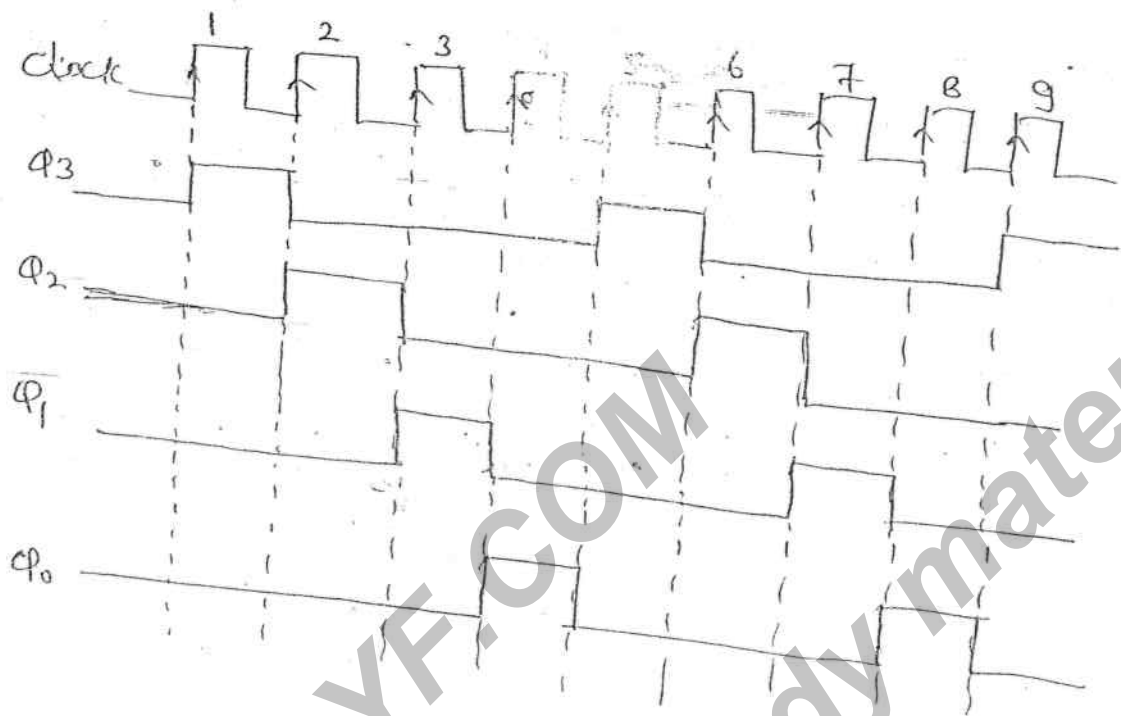


Truth table

clk	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0
1	1	0	0	0
2	0	1	0	0
3	0	0	1	0
4	0	0	0	1
5	1	0	0	0
6	0	1	0	0
7	0	0	1	0

$\rightarrow$ P/4

$\Rightarrow$ In ring counter with n ff's max. states is n .



- In Ring Counter with n flip-flops maximum possible states are n .
- No. of unused states $2^n - n$ ($\because$ total No. of states $= 2^n$).
- In Ring Counter if clock frequency is f then the output frequency at any flip flop is f/n .
- Ring Counter is used in Stepper motor control and SAR type A to D converter.
- phase shift $= 360^\circ/n$.
- In synchronous counter Time period of clock is.

$$T_{CLK} \geq t_{pdff}$$

$$\Rightarrow f_{CLK} \leq \frac{1}{t_{pdff}}$$

$$\Rightarrow f_{max} = \frac{1}{t_{pdff}}$$

$$\text{Duty cycle} = \frac{1}{1+3} = 25\%$$

$$= \frac{t_{on}}{t_{on} + t_{off}}$$